

F:INORY: 온톨로지 기반 금융상품 질의응답 에이전트

신뢰 가능한 금융상품 탐색을 위한 의미 기반 Agent 시스템

ORY

한은규 이정민 김태연

제10회 2026 미래에셋증권 AI Festival

2026년 9월

초록

본 제안서는 국내채권, 국내·해외 ETP, 공모펀드 등 이질적인 금융상품 데이터를 공통 의미 체계로 통합하고, 사용자의 자연어 질의를 온톨로지에 기반하여 해석한 뒤 관계 탐색, 수치 연산, 의미 검색을 적절한 저장소에 분배하여 검증 가능한 근거와 함께 답변하는 금융상품 Agent를 제안한다. 제안 시스템은 Canonical Data Model, Ontology, PostgreSQL, Knowledge Graph, Semantic Search, HyperCLOVA X를 결합하며, 검색 결과를 직접 생성 모델에 전달하지 않고 Evidence Validation을 거치는 Evidence-First 구조를 채택한다. 또한 데이터 범위나 비교 가능성이 충분히 검증되지 않은 경우 추정 답변을 생성하지 않는 Fail-Closed 전략을 통해 금융 도메인에서 요구되는 정확성, 재현성, 설명 가능성을 확보하는 것을 목표로 한다.

주요어: 금융상품 Agent, Ontology, Knowledge Graph, Federated Query, Entity Resolution, Evidence Validation, HyperCLOVA X

목차

1 제안 개요	5
1.1 제안 배경 및 문제 인식	5
1.2 제안 목표	6
1.3 핵심 아이디어	6
1.4 기존 방식과의 차별점	7
2 문제 정의	8
2.1 금융상품 데이터의 분산 및 이질성	9
2.2 상품 유형별 데이터 구조의 차이	9
2.3 의미적 불일치와 Entity/Relation 정합성 문제	10
2.4 LLM 단독 질의응답의 한계	11
2.5 Ontology 기반 의미 통합의 필요성	11
3 제안 시스템 개요	12
3.1 전체 Architecture	13
3.2 Ontology Semantic Layer	14
3.3 오프라인 Knowledge Construction	15
3.4 Canonical Data Model과 Ontology 기반 데이터 통제	17
3.5 Federated Retrieval 및 Evidence 기반 응답 생성	18
3.6 API / Service Layer	20
4 금융상품 지식 모델링 및 Ontology 설계	20
4.1 Canonical Data Model	21
4.1.1 원천 데이터 구성	21
4.1.2 Canonical Model의 구성 영역	22
4.1.3 공통 Entity identity	22
4.1.4 Canonical 금융상품 집계	22
4.1.5 Canonical Fact와 수치 관측	23
4.1.6 원문 Evidence 추적	24
4.1.7 불확실성과 충돌의 명시적 보존	24
4.1.8 Ontology와의 역할 구분	24
4.2 금융상품 Ontology 설계	24
4.2.1 Module과 Class 구조	25
4.2.2 제약과 실행 범위	26
4.3 주요 Entity	26
4.4 주요 Relation과 Graph Projection	27
4.5 금융 지표 및 속성 모델링	29
4.6 식별자 모델 및 Entity Resolution	31
4.7 Data-Ontology Mapping	32
4.8 외부 데이터 및 Provenance 관리	34
5 Agent 기반 질의 처리	35
5.1 Query Understanding	35
5.2 Entity Resolution	36

5.3	Ontology Grounding	36
5.4	Query Planning	37
5.5	RDB / Graph / Semantic Retrieval	38
5.5.1	RDB Retrieval.	38
5.5.2	Graph Retrieval.	38
5.5.3	Semantic Retrieval.	38
5.6	Evidence Construction	39
5.7	Validation	39
5.8	HyperCLOVA X 기반 최종 응답 생성	39
6	주요 질의 유형 및 사용자 시나리오	40
6.1	조건 기반 금융상품 검색	40
6.2	지표 기반 Ranking	41
6.3	특정 기업 관련 금융상품 탐색	42
6.4	ETF 구성종목 기반 탐색	42
6.5	관계 기반 복합 질의	42
6.6	자연어 설명형 질의	42
6.7	데이터 부족·비교 불가 질의 처리	43
6.7.1	필요한 데이터가 존재하지 않는 경우	43
6.7.2	비교 기준이 불명확한 경우	43
6.7.3	지표 제약이 충족되지 않는 경우	43
6.7.4	지원하지 않는 의미가 포함된 경우	44
7	신뢰성 및 검증	44
7.1	Source Provenance	45
7.2	Snapshot 및 Version 관리	45
7.3	Ontology–Data 정합성	46
7.4	PostgreSQL–Neo4j 정합성	46
7.5	수치 및 Ranking 검증	47
7.6	Evidence Validation	48
7.6.1	Query–Plan Consistency.	48
7.6.2	Plan–Execution Consistency.	48
7.6.3	Execution–Evidence Consistency.	48
7.7	Fail-Closed 전략	48
7.8	검증 근거와 범위	49
8	기대 효과 및 확장성	49
8.1	금융상품 탐색 편의성	50
8.2	근거 기반 답변 신뢰도 향상	50
8.3	금융상품 간 의미적 연결	51
8.4	신규 금융상품 및 데이터 공급자 확장	52
8.5	실시간 데이터 연계	52
8.6	데이터 품질 분석 및 보완 가능성	53
8.7	사용자 맞춤형 Agent로의 발전	54
9	결론	55

1. 제안 개요

1.1 제안 배경 및 문제 인식

최근 금융서비스는 단순한 상품 정보 제공을 넘어, 여러 출처에 분산된 금융 데이터를 통합하고 사용자의 자연어 질의에 맞는 정보를 제공하는 지능형 서비스로 확장되고 있다. 특히 금융상품의 종류와 데이터가 빠르게 증가하면서, 사용자가 개별 상품과 지표를 직접 검색하는 방식만으로는 자신에게 필요한 정보를 효율적으로 탐색하기 어려워지고 있다.

동시에 금융권의 AI 활용 방식도 단순한 생성형 AI 기반 질의응답에서 목표를 해석하고 필요한 작업을 분해한 뒤 판단과 실행을 연속적으로 수행하는 **Agentic AI** 중심으로 확장되고 있다. 디지털데일리가 소개한 글로벌 금융서비스 분야 조사에 따르면, 현재 Agentic AI를 활용하고 있다고 응답한 비율은 24%였으며, 추가로 57%가 2030년까지 도입할 것으로 예상하였다. 이를 합산하면 2030년에는 약 81%의 금융회사가 Agentic AI를 현재 활용하거나 도입할 것으로 전망된다. 또한 국내 금융권에서도 고객상담, 자산관리, 여신심사, 내부통제 및 업무자동화 등 다양한 업무에 AI Agent 적용을 확대하는 움직임이 나타나고 있다 [디지털데일리].

이러한 변화는 금융상품 탐색 역시 사용자가 개별 상품과 지표를 직접 검색하는 방식에서, 자연어로 자신의 요구를 제시하면 시스템이 필요한 정보와 연산을 스스로 식별하고, 여러 데이터 소스를 탐색하여 그 결과와 근거를 함께 설명하는 Agent 기반 방식으로 발전할 필요가 있음을 보여준다.

미래에셋증권의 과제설명자료 역시 금융상품 Agent의 핵심을 정형 금융상품 데이터를 Agent가 스스로 탐색·연산하고, 그 결과를 근거에 기반하여 답변하는 Agent RAG·QA로 제시한다. 이를 위해 상품 도메인 특화 Ontology, RDB·Vector·Graph 기반 KnowledgeBase, Intent Analysis & Retrieval Engine, Evidence 기반 Answer Generator를 주요 구성요소로 제시하고 있다 [과제설명자료, 4쪽].

그러나 금융상품에 대한 자연어 질의를 정확하게 처리하는 것은 일반적인 정보 검색보다 복잡하다. 국내채권, 국내 ETF, 해외 ETF, 공모펀드는 서로 다른 속성과 데이터 구조를 가지며, 하나의 사용자 질의 안에서도 상품 유형, 투자지역, 위험등급, 보수, 수익률과 같은 정형 조건뿐 아니라 기업 간 관계, ETF 편입 관계, 상품 간 비교 및 순위와 같은 서로 다른 형태의 정보 처리가 동시에 요구될 수 있다.

예를 들어 “특정 기업이 발행한 증권을 편입한 ETF 중 순자산이 큰 상품을 알려줘”라는 질의는 단순한 키워드 검색으로 해결하기 어렵다. 먼저 기업과 발행 증권을 식별하고, 해당 증권을 편입한 ETF를 관계 기반으로 탐색한 뒤, 후보 상품의 순자산을 비교 가능한 범위에서 조회하고 정렬해야 한다. 즉 하나의 자연어 질의에도 **의미 이해, Entity Resolution, 관계 탐색, 수치 연산, 근거 검색**이 복합적으로 요구된다.

기술세션자료는 관련 정보가 여러 문서 조각에 나뉘어 검색되는 *Fragmented Retrieval* 문제를 제시하며, 복합 질의에 의미 검색, 관계 탐색 및 수치 연산을 함께 활용할 필요성을 설명한다 [기술세션자료, 19-21쪽]. 상품군별 구조와 의미 충돌의 구체적인 문제는 2장에서 다룬다.

문제는 단순히 검색 범위를 넓히는 것으로 해결되지 않는다. 기술세션자료의 국내 ETF 자체 벤치마크에서는 SQL 실행 성공률이 약 100%임에도 실제 정답률은 45%에 머물렀으며, 주요 실패 원인은 잘못된 컬럼, 값, 조건의 선택이었다. 반면 Ontology를 단순한 프롬프트 설명이 아니라 런타임 식별 과정에 활용했을 때 더 큰 정확도 개선이 나타났고, RDB와 Graph를 결합한 방식이 단일 검색 방식보다 안정적인 결과를 보였다 [기술세션자료, 23쪽]. 이는 금융상품 질의응답에서 중요한 문제가 단순히 실행 가능한 SQL이나 유사한 문서를 찾는 것이 아니라, **사용자의 표현을 올바른 금융 개념, 관계, 데이터 필드에 연결하는 것**임을 보여준다.

또한 금융 도메인에서는 정확한 답변을 생성하는 것뿐 아니라 **근거가 부족한 경우 답변을 생성하지 않는 것** 역시 중요하다. 과제설명자료에서는 제공 데이터로 확인할 수 없는 답변 불가 질의가 평가에 포함되며, 해당 질의에 임의의 답변을 생성하면 감점한다고 명시한다. 반대로 “확인할 수

없음”을 명시하거나 답변에 필요한 조건을 역질문하는 것은 올바른 처리로 인정된다 [과제설명자료, 7쪽]. 또한 모든 답변은 데이터에 근거해야 하며, 근거 없는 수익률 전망이나 단정적인 투자 추천은 금지된다 [과제설명자료, 6쪽].

따라서 본 제안이 해결하고자 하는 핵심 문제는 단순히 더 많은 금융상품을 검색하는 것이 아니다. 본 제안은 **자연어 질의를 금융 도메인의 명시적인 의미 체계에 Grounding하고, 질문에 필요한 관계 탐색과 수치 연산을 적절한 데이터 소스에 배분하며, 그 결과를 검증 가능한 Evidence로 구성하는 것을 금융상품 Agent의 핵심 문제로 정의한다.**

이를 위해 본 시스템은 Ontology를 중심 Semantic Layer로 활용하고, RDB, Knowledge Graph, Semantic Search를 결합한 Federated Retrieval과 Evidence Validation을 통해 **검증 가능한 범위에서만 답변하는 신뢰 가능한 Financial Product Analyst**를 제안한다.

1.2 제안 목표

본 제안의 목표는 **이질적인 금융상품 데이터를 공통된 의미 체계로 통합하고, 사용자의 자연어 질의를 Ontology에 기반하여 해석한 뒤, 질의 특성에 가장 적합한 데이터 저장소를 선택·조합하여 검증 가능한 근거와 함께 답변하는 금융상품 Agent를 구축하는 것이다.**

이를 위해 본 시스템은 단순한 LLM 기반 질의응답이나 단일 검색 시스템을 지향하지 않는다. 금융상품 데이터의 구조와 의미를 보존하는 Canonical Data Model을 구축하고, 그 위에 금융상품의 개념·속성·관계를 정의하는 Ontology를 배치한다. 이후 사용자의 질의를 의미 단위로 분석하여 관계 정보는 Graph, 정량적 조건과 연산은 RDB, 자연어 의미 및 설명 정보는 Semantic Search가 담당하도록 역할을 분리한다. 이는 기술세션자료에서 제시한 “Graph는 관계를 찾고, RDB는 숫자를 필터링하며, Vector는 의미를 검색한다”는 Federated Query의 역할 분담과 일치한다 [기술세션자료, 22쪽].

본 제안은 다음 네 가지 목표를 갖는다.

1. 금융상품 데이터의 의미적 통합

상품군별 서로 다른 원천 스키마를 그대로 Agent에 노출하는 대신, Canonical Data Model과 Ontology를 이용하여 동일한 개념과 관계를 일관된 의미로 표현한다.

2. 질의에 따른 동적 검색 및 연산

자연어 질의를 구조화된 실행 계획으로 변환하고, RDB·Graph·Semantic Search 중 필요한 저장소를 선택하여 복합 조건, 관계 탐색, 비교 및 설명을 수행한다.

3. 근거 중심의 신뢰 가능한 응답 생성

검색 결과를 즉시 LLM에 전달하지 않고, 출처, 데이터 기준, 엔티티 정합성, 수치 및 관계 조건을 검증하여 Evidence를 구성한 뒤 HyperCLOVA X가 이를 바탕으로 최종 응답을 생성하도록 한다.

4. 불확실성에 대한 보수적 처리

질의에 필요한 개념이나 관계가 지원되지 않거나, 데이터 범위가 부족하거나, 비교 가능성이 검증되지 않은 경우 추정에 의해 답을 생성하지 않고 해당 요청의 한계를 명시한다.

1.3 핵심 아이디어

본 시스템의 핵심 아이디어는 **Ontology를 단순한 설명 문서가 아니라 질의 해석과 검색·검증을 통제하는 런타임 Semantic Layer로 활용하는 것이다.** 기술세션자료는 Ontology를 “무엇이 가능해야 하는가”를 정의하는 TBox에 가까운 개념·규칙 계층, Knowledge Graph를 “실제로 무엇이 있는가”를 저장하는 ABox에 가까운 인스턴스 계층으로 구분한다 [기술세션자료, 4쪽]. 또한 Ontology가 용어 표준화, 관계 기반 추론, 데이터 무결성 검증, LLM 답변의 지식 그래프 Grounding

을 지원할 수 있음을 설명한다 [기술세션자료, 5쪽].

본 제안은 이러한 Semantic Layer를 실제 Agent 실행 과정에 연결하기 위해 다음 네 단계의 구조를 사용한다.

표 1: 제안 시스템의 핵심 처리 단계

단계	핵심 구성	설명
Define	Canonical Data Model & Ontology	서로 다른 금융상품 원천 데이터를 공통 모델로 정규화하고, 금융상품 도메인의 개념·속성·관계와 의미 제약을 정의한다.
Connect	Knowledge Construction	정규화된 금융상품과 외부 데이터를 Entity Resolution을 통해 동일한 실제 개체에 연결하고, 검증된 관계를 Knowledge Graph에 구성한다. 각 데이터에는 출처와 기준 시점 등의 Provenance를 함께 관리한다.
Retrieve	Ontology-Grounded Federated Query	사용자 질의를 Query Understanding, Entity Resolution, Ontology Grounding, Query Planning 과정을 통해 구조화한다. 수치·조건·정렬에는 RDB, 다단계 관계 탐색에는 Graph, 자연어 의미 탐색에는 Semantic Search를 활용한다.
Reason	Evidence-First Answer Generation	각 저장소의 결과를 바로 답변으로 변환하지 않고, Evidence Builder와 Validator를 통해 출처, 엔티티, 관계, 수치 및 비교 조건을 검증한다. 최종적으로 검증된 Evidence만 HyperCLOVA X에 제공한다.

이 흐름은 기술세션자료가 전체 구조를 **Define** → **Connect** → **Retrieve** → **Reason**으로 정리한 방향과 대응한다. 본 시스템에서는 이 가운데 Reason을 근거 검증과 답변 생성으로 구분하여, 이후 절에서 Validate와 Answer 단계로 구체화한다. 즉 Ontology로 개념과 관계를 정의하고, Knowledge Graph로 실제 데이터를 연결하며, Federated Query로 질문에 필요한 근거를 찾고, LLM/GraphRAG가 근거를 바탕으로 답변한다 [기술세션자료, 24쪽].

따라서 본 시스템에서 LLM은 금융 지식의 유일한 저장소나 판단 주체가 아니라, **구조화되고 검증된 금융 지식을 사용자가 이해할 수 있는 언어로 연결하는 추론 및 표현 계층**으로 동작한다.

1.4 기존 방식과의 차별점

본 제안의 차별점은 단순히 RDB, Graph, Vector DB와 LLM을 함께 사용한다는 데 있지 않다. 핵심은 **하나의 Ontology와 Canonical Data Model을 중심으로 각 저장소의 역할을 명확히 분리하고, 자연어 질의를 이 의미 체계에 Grounding한 후 검증 가능한 실행 계획을 생성한다**는 것이다.

다음 표는 각 접근의 대표적인 기본 구성과 본 시스템의 구현 선택을 비교한 것이다. 다른 RAG나 RDB 시스템도 별도의 식별·검증 계층을 추가할 수 있으므로, 특정 기술의 본질적인 한계나 실측 성능의 우열을 의미하지 않는다.

기술세션자료는 같은 LLM을 사용하더라도 문서 Chunk 중심 접근과 Ontology·Graph·RDB를 함께 사용하는 Semantic AI 접근은 지식 활용 방식이 다르다고 설명한다. 전자는 Embedding으로 관련 문서 조각을 검색하는 데 초점을 두는 반면, 후자는 Ontology 해석, Graph 탐색, RDB 검증·필터를 거쳐 의미·관계·검증 중심으로 답변을 구성한다 [기술세션자료, 16쪽].

본 시스템은 이러한 방향을 실제 실행 구조로 구체화하여 **Ontology를 Query Planning과 Validation의 공통 기준으로 사용한다**. 즉 Ontology는 단순히 LLM 프롬프트에 금융 용어 설명을 제공하는 보조 자료가 아니라,

표 2: 대표적인 기본 구성과 제안 시스템의 설계 비교

구분	문서 중심 RAG	단일 RDB / NL2SQL	제안 시스템
지식 표현	문서 · Chunk 중심	테이블 · 컬럼 중심	Canonical Model + Ontology
자연어 의미 해석	Embedding 유사도 중심	Schema / Column Mapping 중심	Ontology Grounding
정량 필터 · 정렬	별도 정형 연산 결합	SQL 연산	허용 지표 · 출처 계약에 따른 연산
다단계 관계 탐색 설명 · 의미 검색 복합 질의	관계 구조화 필요 강점 검색 Chunk 결합	JOIN 또는 재귀 질의 제한적 SQL 표현 범위에 의존	허용된 Graph 경로 탐색 Semantic Search 활용 Query Decomposition + Federated Query
엔티티 통합	문서 식별 · 연결 구성	Key와 식별 규칙	Scheme · Namespace · grain 검증
답변 근거 의미적 제약 데이터 부족 처리	검색 Chunk 제한적 검색 · 생성 정책에 따름	Query 결과 DB Schema 중심 조회 · 응용 계층 정책에 따름	검증된 Evidence Bundle Ontology 기반 의미 제약 원인별 Validation + Safe Response
LLM 역할	검색 및 생성의 중심	NL2SQL 생성	검증된 근거에 대한 추론 · 설명

- 자연어 질의가 무엇을 의미하는지 결정하고,
- 어떤 관계와 필드를 사용할 수 있는지 제한하며,
- 어떤 저장소에서 근거를 찾아야 하는지를 결정하고,
- 검색된 결과가 의미적으로 유효한지 검증하는

Agent의 런타임 Semantic Layer로 활용된다. 기술세션자료 역시 Ontology를 프롬프트 설명이 아니라 런타임 식별에 사용할 때 더 큰 성능 개선이 관찰되었으며, 이를 “문서가 아니라 런타임 통제 계층”으로 정리한다 [기술세션자료, 23쪽].

또한 본 제안은 “가능한 한 많은 답을 생성하는 시스템”보다 **검증할 수 있는 범위에서 정확하게 답하는 시스템**을 지향한다. 데이터에 존재하지 않는 정보, 의미적으로 지원되지 않는 관계, 신뢰할 수 없는 비교 조건에 대해서는 LLM이 임의로 보완하지 않도록 하며, 이러한 Fail-Closed 전략을 통해 금융상품 탐색에서 요구되는 신뢰성과 설명 가능성을 확보한다. 이는 평가 데이터로 확인할 수 없는 질문에 답변을 생성하면 감점한다는 과제의 평가 원칙과도 직접 연결된다 [과제설명자료, 7쪽].

결과적으로 본 제안은 단순한 Question → Retrieval → LLM 구조를 넘어,

**Question → Semantic Interpretation → Structured Planning → Federated Retrieval
→ Evidence Validation → Grounded Answer**

의 전체 파이프라인을 구축한다. 이는 금융 데이터의 의미와 관계를 이해하고, 필요한 연산을 수행하며, 그 결과를 검증 가능한 근거와 함께 설명하는 **Financial Product Analyst**를 구현하기 위한 접근이다.

2. 문제 정의

금융상품 질의응답은 단순한 문서 검색이나 자연어 생성만으로 해결하기 어려운 문제이다. 금융상품 정보는 여러 상품군과 데이터 공급원에 분산되어 있으며, 상품마다 데이터 구조와 의미 체계가

다르고, 하나의 자연어 질의가 여러 Entity, Relation, Metric 및 조건을 동시에 포함할 수 있기 때문이다.

특히 금융 도메인에서는 단순히 관련된 정보를 찾아내는 것뿐 아니라 **질문의 의미를 정확하게 해석하고, 서로 다른 데이터에서 동일한 금융 개념을 식별하며, 검색 결과가 실제 질문의 모든 조건을 만족하는지 검증하는 것이 중요하다.**

본 절에서는 이러한 문제를 데이터 구조, 의미 정합성, 자연어 질의 처리라는 세 관점에서 정의한다.

2.1 금융상품 데이터의 분산 및 이질성

금융상품에 대한 하나의 질문에 답하기 위해 필요한 정보는 항상 하나의 데이터셋에 존재하지 않는다. 상품 기본정보, 운용사, 발행사, 기초지수, 구성종목, 수익률, 순자산, 보수, 투자전략 등은 서로 다른 데이터 Source에서 제공될 수 있으며, 일부 정보는 정형 테이블로, 일부는 자연어 설명이나 외부 자료의 형태로 존재한다.

미래에셋증권의 과제설명자료에서도 금융상품 Agent가 정형 및 비정형 데이터를 함께 분석하고, RDB, Vector, Graph를 결합한 KnowledgeBase를 사용해야 함을 명시하고 있다 [과제설명자료, 4 쪽].

문제는 단순히 데이터가 여러 곳에 있다는 점에 그치지 않는다. 각 Source는 서로 다른

- Schema와 Field Name,
- Entity Identifier,
- 값의 표현 방식,
- 데이터 기준 시점,
- 데이터 Grain,
- 결측 및 갱신 정책

을 가질 수 있다.

예를 들어 동일한 운용사도 어떤 데이터에서는 정식 법인명으로, 다른 데이터에서는 약칭으로 표현될 수 있으며, 같은 금융상품의 수치 정보도 관측 시점이나 단위가 다르면 직접 비교할 수 없다. 따라서 금융상품 질의응답의 첫 번째 문제는

분산된 금융 데이터를 동일한 의미 기준에서 해석할 수 있는가

로 정의할 수 있다.

이는 단순히 여러 파일을 하나의 테이블에 병합하는 문제와 다르다. 원천별 의미와 기준 시점을 유지하면서도 질의 처리 단계에서는 동일한 금융 개념으로 접근할 수 있는 공통 의미 체계가 필요하다.

2.2 상품 유형별 데이터 구조의 차이

국내채권, 국내 ETF, 해외 ETF, 공모펀드는 모두 금융상품이라는 상위 범주에 속하지만, 실제로 의미 있는 속성과 데이터 구조는 서로 다르다.

예를 들어 채권에서는 발행사, 만기, 신용등급, 수익률과 같은 정보가 중요하지만, ETF에서는 운용사, 기초지수, 순자산, 총보수, 투자지역 및 자산 유형과 같은 정보가 중요하다. 공모펀드 역시 Fund와 Fund Class의 구분, 판매 형태, 운용전략 등 ETF나 채권과 다른 데이터 구조를 가질 수 있다.

따라서 모든 상품을 동일한 Flat Schema에 강제로 맞추면 두 가지 문제가 발생한다.

첫째, 특정 상품군에서만 의미가 있는 정보를 다른 상품군에도 동일하게 적용하는 **Semantic Overgeneralization**이 발생할 수 있다.

둘째, 서로 다른 Grain의 데이터를 하나의 상품 Record로 간주하면서 원래 데이터가 표현하고 있던 관계가 손실될 수 있다.

예를 들어 동일한 “상품” 데이터 안에서도

Fund $\neq$ Fund Share Class

일 수 있으며, 이를 구분하지 않으면 Class 수준의 속성을 Fund 자체의 속성으로 잘못 해석할 수 있다.

즉 문제는 모든 금융상품이 공통적으로 갖는 속성을 찾는 것뿐 아니라,

공통 속성과 상품 유형별 고유 의미를 동시에 보존하는 표현을 정의하는 것

이다.

따라서 금융상품 데이터 모델은 하나의 통합 Schema를 만드는 것보다, 공통적인 상위 개념과 상품군별 특화 속성 및 관계를 함께 표현할 수 있어야 한다.

2.3 의미적 불일치와 Entity/Relation 정합성 문제

데이터의 구조가 동일하더라도 문자열이 같다는 이유만으로 동일한 의미를 가진다고 볼 수는 없다.

금융 데이터에서는 하나의 표현이 문맥에 따라 서로 다른 의미를 가질 수 있고, 반대로 서로 다른 표현이 동일한 실제 Entity를 가리킬 수도 있다.

예를 들어 “채권”이라는 단어는 하나의 금융상품 유형인 Bond를 의미할 수도 있고, ETF가 투자하는 Asset Class로서 Bond를 의미할 수도 있다.

즉,

$$\text{“채권”} \rightarrow \begin{cases} \text{Financial Product: Bond} \\ \text{Asset Type: Bond} \end{cases}$$

와 같이 Context에 따라 다른 의미를 가진다.

Entity 수준에서도 같은 문제가 존재한다. 기업명, 운용사명, 지수명, 상품명에는 정식 명칭, 약칭, 영문명, 종목코드 등 다양한 표현이 존재하기 때문에 단순 문자열 Matching만으로는 안정적인 식별이 어렵다.

Relation은 더 강한 의미 제약을 요구한다. 예를 들어

managedBy, issuedBy, tracksIndex

와 같은 관계는 어떤 종류의 Entity 사이에 연결되는지에 따라 의미가 결정된다.

따라서 관계가 데이터에 존재한다는 사실만으로 충분하지 않고,

Subject Type + Relation Type + Object Type

의 조합이 금융 도메인에서 허용되는 관계인지 확인해야 한다.

이를 검증하지 않으면 원천 데이터의 Field를 잘못 해석하여 존재하지 않는 관계를 Knowledge Graph에 생성하거나, 서로 다른 실제 Entity를 하나로 병합하는 오류가 발생할 수 있다.

결국 금융상품 Agent가 해결해야 할 문제는 문자열 수준의 Matching이 아니라,

Natural Language Expression $\rightarrow$ Canonical Entity / Concept $\rightarrow$ Valid Relation

의 의미적 정합성을 유지하는 것이다.

2.4 LLM 단독 질의응답의 한계

LLM은 자연어 질의를 이해하고 복잡한 내용을 사용자에게 설명하는 데 강점을 가지지만, 금융상품 데이터에 대한 정확한 질의응답을 LLM의 Parametric Knowledge에만 의존하기는 어렵다.

첫 번째 문제는 **최신성과 데이터 범위**이다. 금융상품의 순자산, 수익률, 보수, 구성종목 등의 정보는 지속적으로 변경되며, LLM이 학습 과정에서 습득한 정보가 현재 평가 데이터와 일치한다고 보장할 수 없다.

두 번째는 **정확한 연산과 조건 처리** 문제이다. 사용자의 질문에는 다음과 같은 조건이 포함될 수 있다.

“국내 상장 ETF 중 미국 주식에 투자하는 상품을 순자산이 큰 순서로 3개 알려줘.”

이 질의를 정확히 처리하기 위해서는 ETF 여부, 투자 지역, 자산 유형, 순자산, 정렬 방향, 결과 개수를 모두 보존해야 한다.

LLM이 이 조건을 자유롭게 해석하여 직접 결과를 생성하면 일부 조건을 누락하거나, 사용자가 요청하지 않은 조건을 추가하거나, 정렬 기준을 잘못 적용할 가능성이 있다.

세 번째는 **근거 존재 여부와 답변 가능 여부의 구분**이다. LLM은 주어진 Context에 충분한 정보가 없더라도 언어적으로 자연스러운 답변을 생성할 수 있다. 그러나 금융상품 질의에서는 정보가 존재하지 않는 상황에서 그럴듯한 추론을 생성하는 것보다 “현재 데이터로 확인할 수 없음”을 반환하는 것이 더 중요할 수 있다.

과제설명자료 역시 검색된 Evidence에 기반한 답변을 요구하며, 데이터에 없는 내용을 생성하지 않아야 함을 명시하고 있다 [과제설명자료, 4쪽]. 또한 평가 과정에서도 근거가 없는 질문에 임의의 답변을 생성하는 것보다 확인할 수 없음을 명시하거나 필요한 조건을 요청하는 처리를 요구한다 [과제설명자료, 7쪽].

따라서 본 문제에서 LLM은

Knowledge Source

자체가 아니라,

Structured Knowledge를 이해하고 설명하는 Language Interface

로 사용하는 것이 적절하다.

2.5 Ontology 기반 의미 통합의 필요성

앞서 정의한 문제들은 데이터를 하나의 저장소에 모으는 것만으로는 해결되지 않는다.

핵심 문제는 서로 다른 데이터 Source와 자연어 질의가 **동일한 금융 개념을 동일한 방식으로 해석하는가**에 있다.

이를 위해서는 다음 요소를 명시적으로 정의할 필요가 있다.

- 어떤 금융상품과 Entity가 존재하는가
- 각 Entity가 어떤 속성을 가질 수 있는가
- Entity 사이에 어떤 Relation이 허용되는가
- 자연어의 다양한 표현이 어떤 개념을 의미하는가
- 특정 Field와 Metric이 어떤 금융 의미를 가지는가
- 어떤 조합의 Entity와 Relation이 의미적으로 유효한가

Ontology는 이러한 정보를 공통 Semantic Contract로 정의한다.

기술세션자료에서는 Ontology를 실제 Entity와 Fact를 저장하는 Knowledge Graph와 구분하여, Ontology는 “무엇이 존재할 수 있고 어떤 관계가 가능한가”를 정의하고, Knowledge Graph는 “실제로 어떤 Entity와 관계가 존재하는가”를 표현한다고 설명한다 [기술세션자료, 11-12쪽].

이 구분은 금융상품 Agent에서 특히 중요하다.

예를 들어 데이터에

ETF A -- managedBy --> Company B

라는 관계가 존재한다고 해서 문자열 형태 그대로 신뢰하는 것이 아니라, Ontology를 통해

ETF $\subseteq$ FinancialProduct

이며,

managedBy: ETF / Fund $\rightarrow$ Organization

와 같은 의미적 제약을 만족하는지를 확인할 수 있다.

또한 사용자 질의의 표현도 동일한 Ontology Vocabulary로 변환할 수 있기 때문에,

Natural Language $\leftrightarrow$ Ontology $\leftrightarrow$ Canonical Data

의 공통 의미 기준을 형성할 수 있다.

따라서 본 문제에서 Ontology가 필요한 이유는 단순히 Knowledge Graph를 구축하기 위해서가 아니다. Ontology는 서로 다른 원천 데이터, 자연어 질의, RDB Field, Graph Relation, 검색 결과 사이의 의미적 일관성을 유지하기 위한 Semantic Contract로 필요하다.

결국 본 과제의 핵심 문제는 “LLM이 금융 질문에 답할 수 있는가”가 아니라,

자연어 금융 질의를 명시적인 금융 의미로 변환하고, 서로 다른 데이터 Source에서
그 의미를 만족하는 검증 가능한 Evidence를 찾을 수 있는가?

로 정의할 수 있다.

이 문제 정의를 바탕으로 다음 절에서는 Ontology를 중심으로 금융상품 데이터를 통합하고, 여러 Knowledge Source를 질의 특성에 따라 활용하는 제안 시스템의 구조를 설명한다.

3. 제안 시스템 개요

본 시스템은 금융상품 데이터를 단순히 하나의 데이터베이스에 적재한 뒤 LLM이 직접 탐색하도록 하는 구조가 아니다. 금융상품의 개념, 속성 및 관계를 Ontology를 통해 명시적으로 정의하고, 사용자 질의를 해당 의미 체계에 Grounding한 후 질의 특성에 따라 RDB, Knowledge Graph, Semantic Search를 조합하여 검증 가능한 Evidence를 구성한다. 최종적으로는 검증된 Evidence를 기반으로 설정된 HyperCLOVA X 답변기 또는 결정적 Evidence 답변기가 사용자에게 전달할 응답을 생성한다.

미래에셋증권 기술세션자료는 복합 금융 질의를 의미 단위로 분해한 뒤 Graph, RDB, Vector 등 서로 다른 검색 수단에 분배하고, 검색 결과를 통합·검증하여 답변하는 Federated Query Architecture를 제시한다. 특히 Graph는 관계 탐색, RDB는 수치 필터링과 연산, Vector 기반 검색은 자연어 의미 탐색에 각각 강점을 가진다고 설명한다 [기술세션자료, 21-22쪽].

본 시스템은 이러한 방향을 Ontology-Grounded Federated Agent Architecture로 구체화하였다. 전체 구조는 사전 지식 구축, Ontology 기반 의미 통제, 온라인 Agent 실행, Federated Retrieval, Evidence 기반 응답 생성 과정으로 구성된다.

3.1 전체 Architecture

그림 1은 제안 시스템의 전체 실행 구조를 나타낸다.

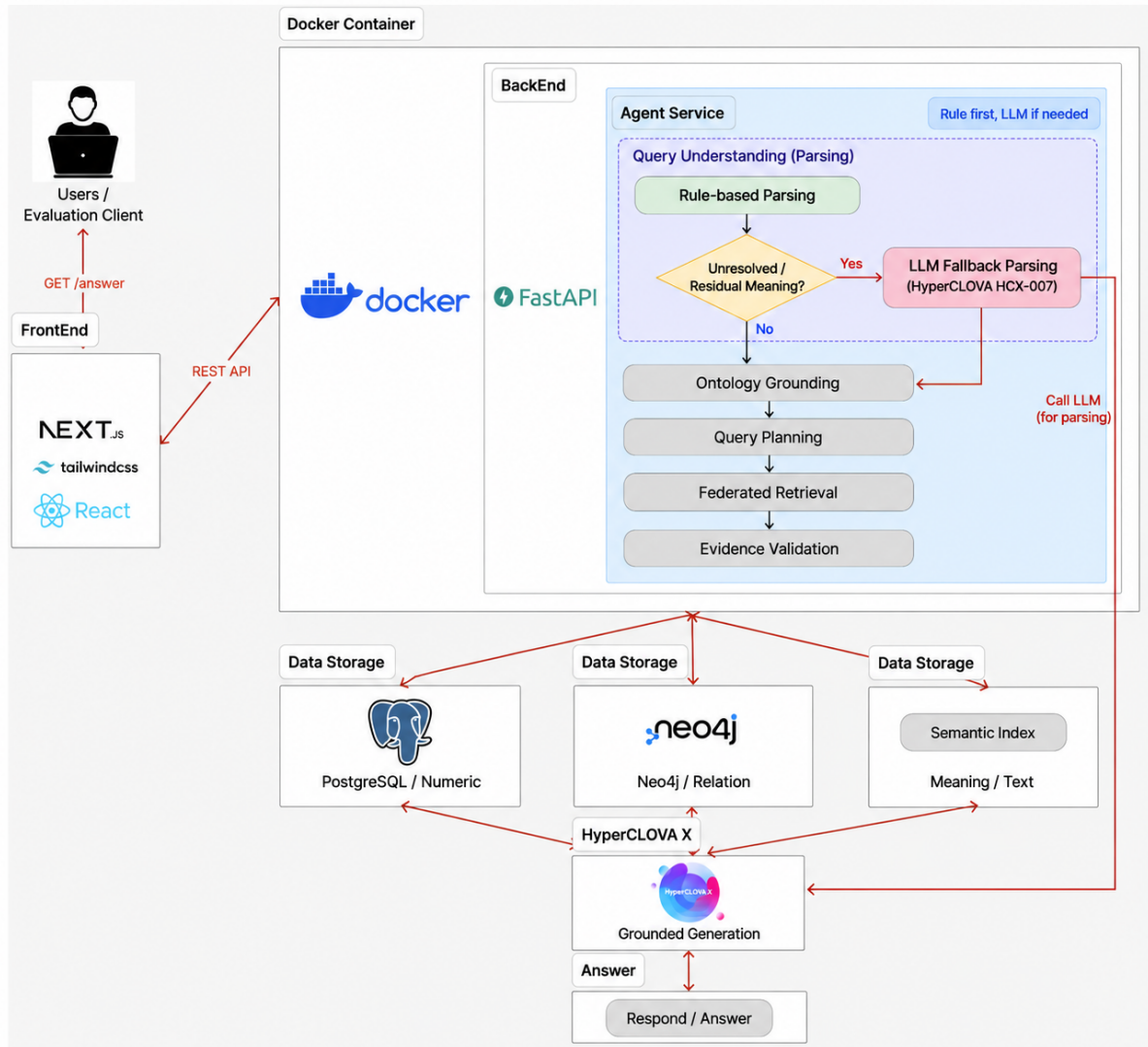


그림 1: 제안 금융상품 Agent의 전체 시스템 아키텍처

사용자는 Next.js, React, Tailwind CSS 기반의 FrontEnd를 통해 금융상품 관련 자연어 질의를 입력한다. 질의는 REST API를 통해 FastAPI 기반 BackEnd의 Agent Service로 전달되며, Agent Service는 질의를 다음과 같은 구조화된 단계로 처리한다.

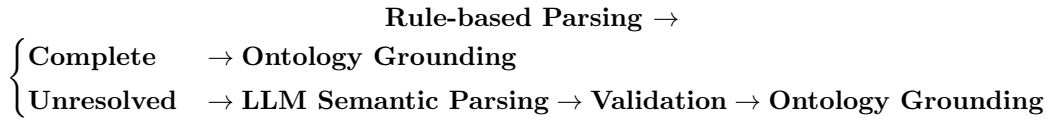
Query Understanding → Ontology Grounding → Query Planning → Federated Retrieval → Evidence Validation

Query Understanding 단계에서는 사용자 질문에서 상품 유형, 조건, 관계, 정렬 기준 및 요청 정보를 추출한다. 본 시스템은 모든 질의를 처음부터 LLM에 전달하지 않고, 먼저 결정적 Rule-based Parser를 이용하여 해석한다. 규칙만으로 질의의 의미와 조건을 충분히 구조화할 수 있는 경우에는 LLM 호출 없이 이후 단계로 진행한다.

반면 규칙 기반 해석 이후에도 해석되지 않은 표현이 남거나, 질문 전체의 의미를 종합적으로 해석해야 하는 경우에는 HyperCLOVA X 기반 Semantic Parser를 fallback으로 호출한다. LLM은 질문 원문과 규칙 기반 분석 결과를 바탕으로 후보 Semantic Representation을 생성하며, 생성된

결과는 사전에 정의된 Schema와 시스템의 Semantic Contract를 통해 검증된 경우에만 이후 단계로 전달된다.

따라서 Query Understanding은 다음과 같은 **Rule-first, LLM-if-needed** 구조로 동작한다.



이후 Ontology Grounding을 통해 자연어 표현을 시스템이 사용하는 금융 개념, 관계 및 Field로 변환하고, Query Planning 단계에서는 해당 질의를 어떤 Knowledge Source에서 어떤 순서로 처리할지 결정한다. 이 과정에서 LLM이 직접 SQL이나 Graph Query를 생성하여 실행하는 것이 아니라, 검증된 Semantic Representation을 기반으로 시스템의 결정적 Planner와 Query Compiler가 실제 실행 계획을 구성한다.

Federated Retrieval에서는 정확한 수치 및 조건 검색이 필요한 경우 PostgreSQL, Entity 사이의 관계 탐색이 필요한 경우 Neo4j, 설명형 또는 의미 기반 검색이 필요한 경우 Semantic Index를 활용한다.

$$\begin{array}{ll} \text{PostgreSQL} = \text{Numeric} / \text{Structured Search} & \text{Neo4j} = \text{Relation Search} \\ \text{Semantic Index} = \text{Meaning} / \text{Text Search} & \end{array}$$

복합 질의의 경우 하나의 저장소만 사용하는 것이 아니라, Graph에서 탐색한 Entity를 PostgreSQL의 정량 조건 검색에 연결하거나, Semantic Search 결과를 구조화된 Entity와 연결하는 방식으로 여러 Knowledge Source를 조합하여 처리할 수 있다.

각 저장소에서 반환된 결과는 바로 LLM에 전달되지 않는다. Evidence Validation을 통해 사용자의 질의 조건과 실제 검색 결과가 일치하는지, 요청된 Field와 Entity에 충분한 근거가 존재하는지, 비교 및 정렬에 필요한 단위와 데이터 제약이 만족되는지를 확인한다. 검증을 통과한 Evidence만 최종 Answer Generation 단계에 전달된다.

최종 답변은 설정된 HyperCLOVA X 답변기 또는 결정적 Evidence 답변기가 생성한다. HyperCLOVA X가 활성화된 경우에도 모델이 새로운 금융 사실을 생성하거나 임의로 데이터베이스를 탐색하는 것이 아니라, 앞선 단계에서 검증된 Evidence를 입력으로 받아 사용자에게 전달할 자연어 응답을 구성한다.

따라서 본 시스템에서 HyperCLOVA X는 크게 두 가지 제한된 역할을 수행한다.

1. **Semantic Parsing Fallback:** Rule-based Parser만으로 해석하기 어려운 질의를 구조화된 Semantic Representation으로 변환한다.
2. **Grounded Answer Generation:** Retrieval과 Evidence Validation을 통과한 금융 근거를 자연어 답변으로 변환한다.

즉, 본 시스템에서 LLM은 금융 지식의 유일한 저장소나 데이터 검색 주체가 아니라, **필요한 경우 질의 의미를 보조적으로 해석하고, 검증된 금융 Evidence를 자연어 응답으로 변환하는 제한된 추론 및 생성 계층**으로 사용된다. 실제 금융 데이터의 조회, 관계 탐색, 비교 가능성 판단 및 Evidence 검증은 Ontology, Planner, PostgreSQL, Neo4j, Semantic Index와 결정적 Validation Logic에 의해 수행된다.

3.2 Ontology Semantic Layer

본 시스템의 핵심은 Ontology를 별도의 KnowledgeBase로만 사용하는 것이 아니라 Agent 전반의 의미를 통제하는 **Semantic Layer**로 활용한다는 점이다.

그림 2와 같이 Ontology는 금융상품의 Concept, Relation, Field, Constraint를 정의하고, 이 의미 체계는 Grounding, Planning, Validation의 공통 기준으로 사용된다.

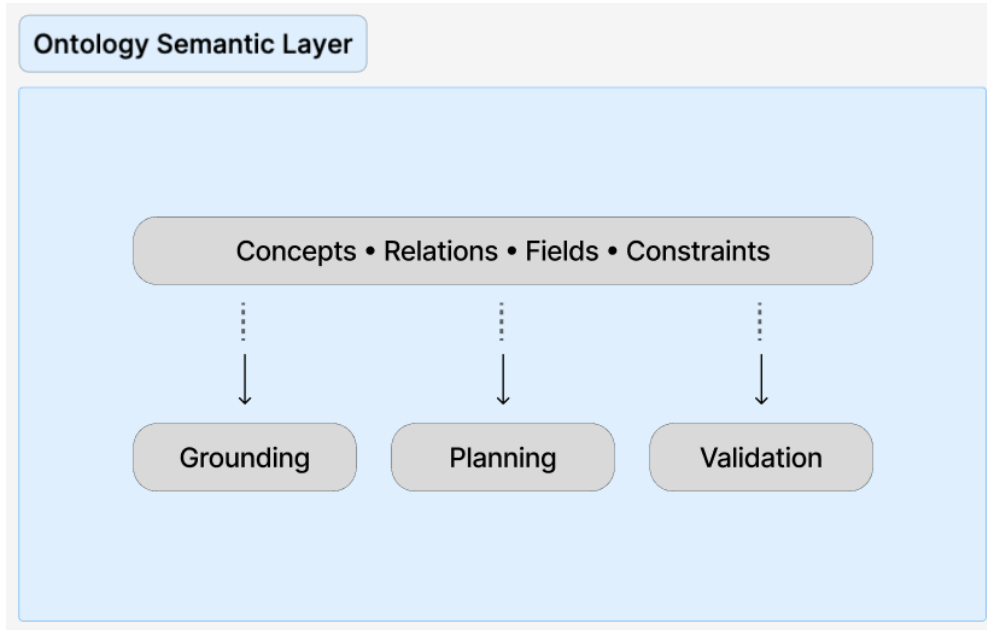


그림 2: Agent 실행을 통제하는 Ontology Semantic Layer

Ontology의 역할은 크게 세 단계에 적용된다.

- **Grounding:** 자연어 표현을 금융상품 Concept, Relation 및 Field에 연결한다.
- **Planning:** Grounding된 의미를 기준으로 어떤 데이터와 Retrieval 방식이 필요한지 결정한다.
- **Validation:** 검색 및 실행 결과가 Ontology가 정의하는 의미와 제약을 만족하는지 검사한다.

예를 들어 사용자가 “순자산”이라고 표현하면 이를 단순 문자열로 데이터베이스 Column과 연결하지 않는다. 먼저 Ontology가 정의하는 AUM의 의미에 연결하고, 이를 Planner가 사용하는 Canonical Field로 변환한 뒤 실제 PostgreSQL의 물리적 Field에 연결한다.

Natural Language → Ontology Meaning → Canonical Field → Physical Storage

이와 같이 의미와 물리 저장 구조를 분리함으로써, 원천 데이터의 Column 이름이나 개별 데이터셋의 표현 방식이 달라지더라도 Agent는 동일한 금융 개념을 일관되게 사용할 수 있다.

또한 Ontology는 Relation의 허용 범위와 의미적 호환성을 정의하기 때문에 잘못된 Entity 사이의 관계를 탐색하거나 사용자 질의를 임의의 금융 Field로 해석하는 것을 제한하는 역할도 수행한다.

이는 기술세션자료에서 Ontology를 Query Decomposition, Entity Resolution, Semantic Grounding 및 Answer Validation을 지원하는 AI Semantic Layer로 설명한 방향과 대응한다 [기술세션자료, 11-14쪽].

3.3 오프라인 Knowledge Construction

온라인 질의 처리 이전에는 금융상품 데이터를 Agent가 일관되게 탐색할 수 있는 형태로 변환하는 사전 Knowledge Construction 과정이 수행된다.

그림 3은 주최 측 금융상품 데이터와 외부 금융 데이터를 KnowledgeBase로 구축하는 전체 흐름을 나타낸다.

사전 지식 구축 과정은 다음 순서로 진행된다.

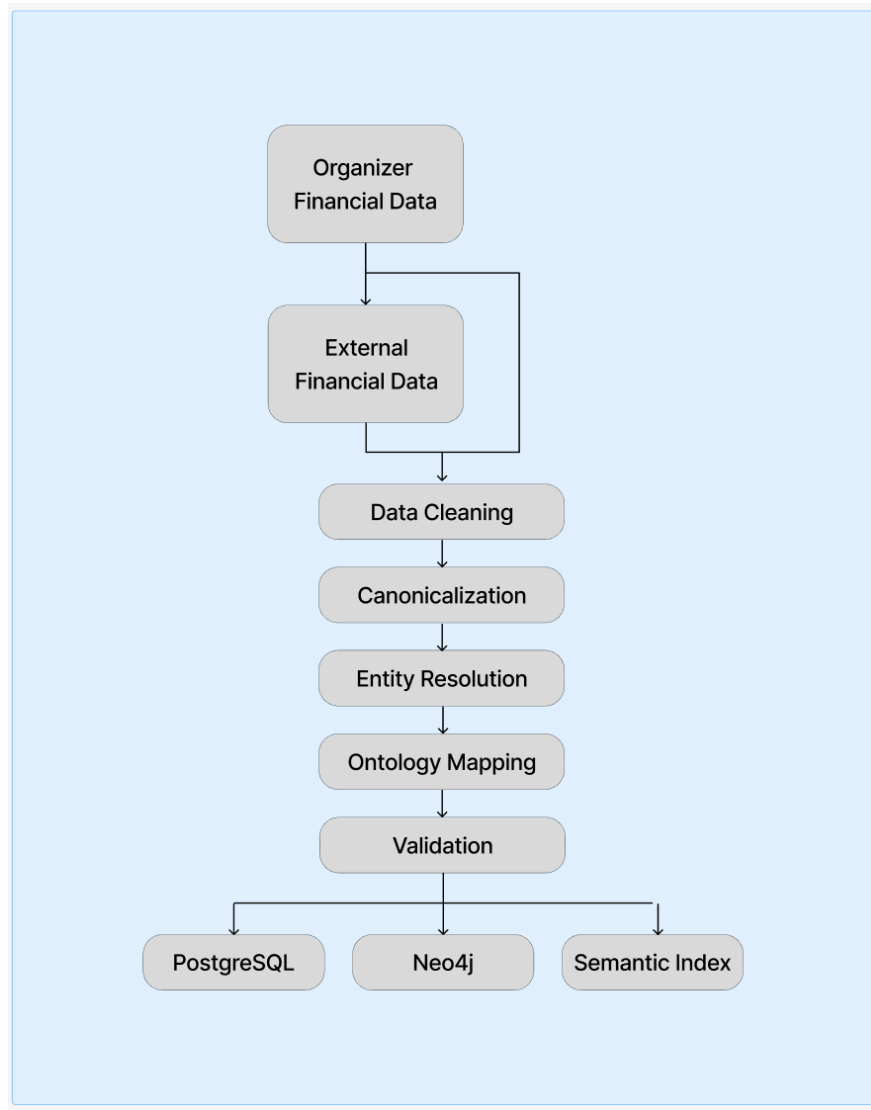


그림 3: 금융상품 데이터의 오프라인 Knowledge Construction 과정

1. Financial Data Ingestion

국내채권, 국내 ETF, 해외 ETF, 공모펀드 등 주최 측에서 제공한 정형 금융상품 데이터를 수집한다. 필요한 경우 금융상품의 관계 및 설명 정보를 보완하기 위한 외부 금융 데이터도 별도의 Source로 관리한다.

2. Data Cleaning

결측값, 비정상 값, 표현상의 차이 등 원천 데이터에서 직접 사용할 수 없는 부분을 정리하되, 원본 정보와 출처는 보존한다.

3. Canonicalization

상품군마다 서로 다른 Field와 값 표현을 공통적인 Canonical Data Model로 변환한다. 이를 통해 서로 다른 데이터셋의 금융상품을 동일한 의미 체계에서 조회할 수 있도록 한다.

4. Entity Resolution

동일한 기업, 상품, 운송사, 발행사 등이 여러 데이터에서 서로 다른 이름 또는 식별자로 표현되는 경우, 이를 Canonical Entity에 연결한다.

5. Ontology Mapping

정규화된 Entity, Metric, Relation을 Ontology의 Class, Property 및 Relation에 연결한다. 의미를 확정할 수 없는 데이터는 임의로 해석하여 새로운 지식을 생성하지 않는다.

6. Validation

생성된 Canonical Data와 Relation이 Ontology 및 데이터 품질 조건을 만족하는지 확인한다.

7. Knowledge Storage

검증된 데이터는 정보의 특성에 따라 PostgreSQL, Neo4j 및 Semantic Index에 적재한다.

이 과정에서 세 저장소는 동일한 데이터를 단순 복제하는 것이 아니라 각자의 역할에 적합한 표현을 담당한다. PostgreSQL은 정형 데이터와 수치 Metric, Neo4j는 Entity 사이의 관계, Semantic Index는 자연어 의미 및 설명 검색을 중심으로 구성한다.

3.4 Canonical Data Model과 Ontology 기반 데이터 통제

원천 데이터를 정규화하는 과정에서 Canonical Data Model과 Ontology는 서로 독립적으로 동작하지 않는다. Canonical Data Model은 Agent가 실제로 탐색하고 연산할 수 있는 공통 데이터 표현을 제공하고, Ontology는 해당 데이터가 가지는 의미와 허용되는 관계를 정의한다.

그림 4는 두 계층이 Knowledge Construction 과정에 어떻게 적용되는지를 나타낸다.

Canonical Data Model과 Ontology Schema는 Canonicalization, Ontology Mapping, Validation 과정에 공통적인 Semantic Guidance를 제공한다.

예를 들어 하나의 상품에 대해 원천 데이터마다 서로 다른 순자산 Field가 존재하더라도, Canonicalization 과정에서는 이를 동일한 Canonical Metric으로 변환한다. 이후 Ontology Mapping에서는 해당 Metric이 어떤 금융 개념을 의미하는지 연결하고, Validation 단계에서는 해당 값이나 Relation이 허용된 Entity Type에 적용되고 있는지 확인한다.

따라서 본 시스템은

Source Representation → Canonical Representation → Semantic Interpretation

의 계층을 명확히 분리한다.

이 구조를 통해 새로운 금융상품이나 데이터 공급자가 추가되더라도 기존 Agent Pipeline 전체를 수정하기보다, 새로운 원천 데이터를 Canonical Data Model과 Ontology에 Mapping하는 방식으로 확장할 수 있도록 설계하였다.

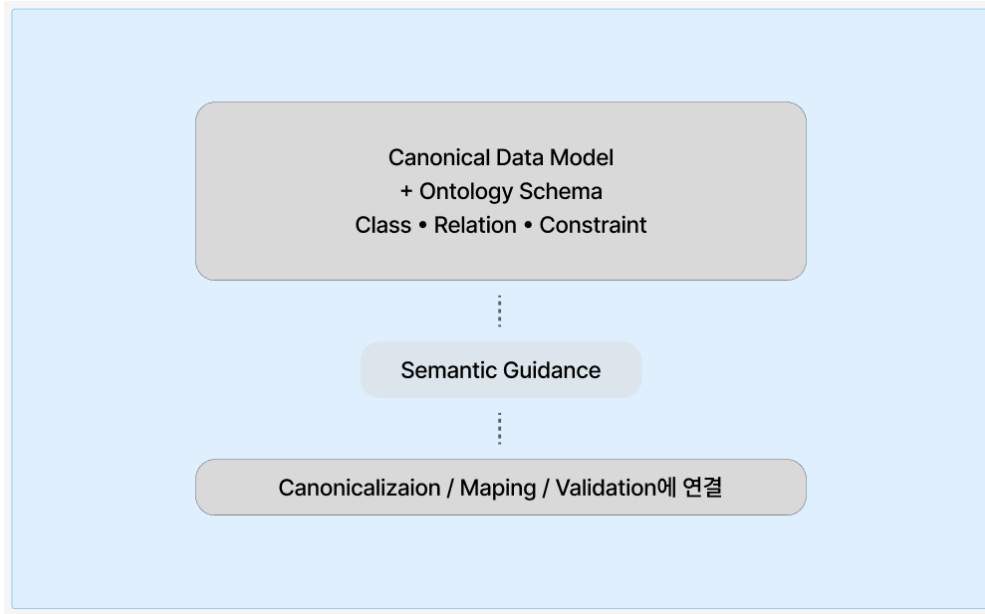


그림 4: Canonical Data Model과 Ontology Schema를 이용한 Semantic Guidance

3.5 Federated Retrieval 및 Evidence 기반 응답 생성

온라인 질의에서는 하나의 Retrieval 방식이 모든 금융상품 질문을 처리하도록 하지 않는다. 질의를 구성하는 의미에 따라 여러 Knowledge Source의 역할을 분리하고, 필요한 경우 두 개 이상의 Retrieval 결과를 조합한다.

그림 5는 Federated Retrieval 이후 최종 응답 생성까지의 흐름을 나타낸다.

Query Planning에서 생성된 실행계획은 질의의 의미에 따라 다음 세 Retrieval 경로로 분배된다.

- **RDB Retrieval:** 수치 조건, 정형 Filter, Sort 및 Ranking과 같이 정확한 연산이 필요한 경우 사용한다.
- **Graph Retrieval:** 상품, 기업, 운송사, 발행사, 증권 등 Entity 사이의 Relation 및 Multi-hop 관계 탐색에 사용한다.
- **Semantic Retrieval:** 자연어 설명, 투자전략, 의미적으로 유사한 표현 등 구조화된 Field만으로 처리하기 어려운 정보 탐색에 사용한다.

각 Retrieval 결과는 하나의 Evidence Builder로 다시 모인다. Evidence Builder는 검색 결과를 단순 나열하는 대신 답변의 근거로 사용할 수 있도록 Entity, Metric, Relation, Source, Snapshot 등의 정보를 구조화된 Evidence로 구성한다.

이후 Evidence Validator는 수집된 근거가 사용자의 원래 질의를 실제로 만족하는지 검사한다. 검색 결과가 존재하더라도 필수 조건이 누락되었거나, 지원되지 않는 의미가 포함되어 있거나, 현재 데이터만으로 충분한 근거를 확보할 수 없는 경우에는 정상 Answer Generation으로 진행하지 않는다.

검증 결과에 따라 실행은 두 경로로 분기된다.

Validated Evidence → Answer Generator → HyperCLOVA X → Final Answer

또는

Unsupported / Insufficient Evidence → Safe Response → Final Answer

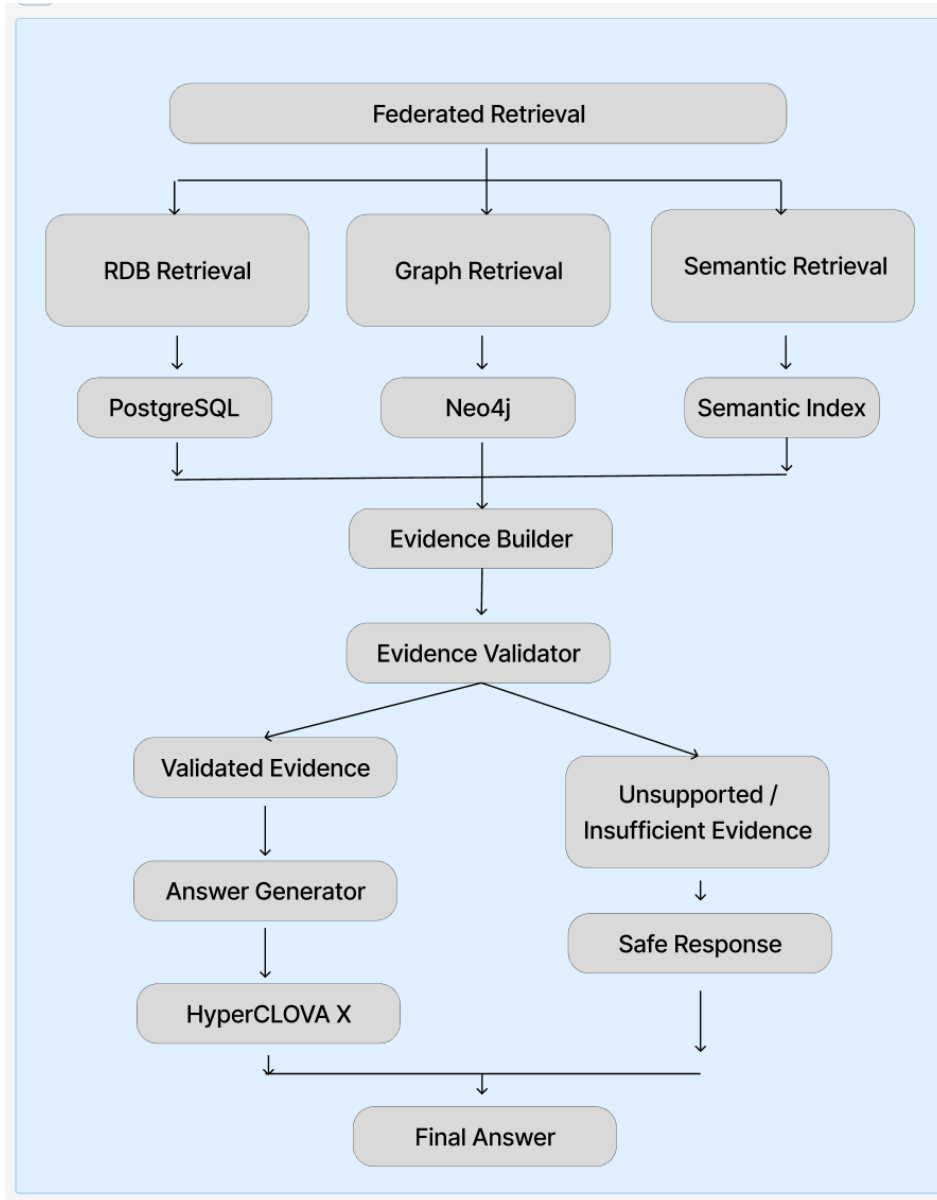


그림 5: Federated Retrieval과 Evidence 기반 최종 응답 생성 흐름

따라서 본 시스템은 지원하지 않는 조건을 제거한 채 나머지 질의만 실행하는 부분적 의미 손실을 허용하지 않는다.

특히 데이터에서 확인할 수 없는 내용에 대해 LLM의 내부 지식이나 추정을 이용하여 임의로 답변하지 않고, 필요한 Evidence가 충분하지 않은 경우에는 확인할 수 없음을 명시적으로 반환한다. 이는 과제설명자료에서 제시한 Evidence 기반 답변 및 환각 방지 원칙과 일치한다 [과제설명자료, 4,7쪽].

3.6 API / Service Layer

Agent Pipeline은 평가 환경에서 안정적으로 호출할 수 있도록 FastAPI 기반 Service로 제공한다.

사용자는 FrontEnd를 통해 질의를 입력할 수 있으며, 평가 시스템은 FrontEnd를 거치지 않고 FastAPI의 GET /answer Endpoint를 직접 호출할 수 있다.

과제 명세에 따라 /answer는 question_id와 question을 입력으로 받고, 다음 다섯 개의 문자열 Field를 반환한다 [과제설명자료, 11쪽].

```
{
  "question_id": "...",
  "question": "...",
  "retrieved_context": "...",
  "think_trace": "...",
  "answer": "..."
}
```

retrieved_context에는 답변 생성에 사용된 검색 근거를 제공하며, think_trace는 모델의 내부 Chain-of-Thought를 노출하는 것이 아니라 질의 해석, Grounding, Retrieval, Validation 등 Agent가 수행한 운영상의 처리 단계와 Tool 사용 결과를 요약하는 용도로 사용한다.

Production 환경은 Docker 기반으로 구성하며, Agent API와 PostgreSQL, Neo4j 등의 실행 Component를 분리된 Service로 관리한다. 애플리케이션 코드는 Docker Image Version으로, Ontology와 외부 데이터 및 Semantic Index는 별도의 Artifact Version과 Checksum으로 관리한다.

이를 통해 동일한 Agent 코드라도 어떤 데이터와 Ontology를 사용하여 실행되었는지를 추적할 수 있으며, 평가 환경에서 재현 가능한 Runtime을 유지하도록 한다.

결과적으로 Service Layer는 단순한 HTTP Interface를 넘어, **Agent 코드, Knowledge Artifact 및 실행 환경을 일관된 상태로 유지하는 Production Boundary** 역할을 수행한다.

4. 금융상품 지식 모델링 및 Ontology 설계

앞 절에서 설명한 Semantic Layer를 실제 금융상품 데이터에 적용하기 위해, 본 프로젝트에서는 국내 ETF/ETN, 해외 ETF/ETN, 공모펀드, 국내채권의 공통 개념과 관계를 하나의 의미 체계로 구조화하였다. 대회에서 제시한 Ontology 구성 가이드라인에 따라 [과제설명자료, 9쪽], 공통 금융 개념과 상품군별 개념을 분리하는 모듈형 구조를 적용하였다.

금융 개념을 정의하는 과정에서는 EDM Council이 공개한 Financial Industry Business Ontology(FIBO)를 함께 참고하였다 [FIBO]. 특히 FIBO의 금융상품 관련 Ontology와 Funds.rdf를 검토하여, 금융상품과 관련 주체를 독립된 Class로 구분하고 Entity 사이의 관계를 명시적으로 표현하는 방식, 그리고 금융 도메인을 여러 Ontology Module로 분리하여 관리하는 방식을 설계에 참고하였다. 다만 FIBO의 Class와 Relation을 그대로 적용한 것은 아니며, 본 과제에서 제공된 데이터 Schema와 실제 질의 범위에 필요한 개념을 중심으로 경량화하여 재구성하였다.

본 프로젝트의 지식 모델에서는 원천 데이터의 물리적 구조, Agent가 사용하는 공통 의미, 그리고

Entity 사이의 관계를 구분하였다. 이에 따라 서로 다른 상품군의 Schema를 직접 하나의 Table로 병합하기보다, Canonical Representation을 통해 의미를 정규화하고 Ontology가 해당 의미의 Class와 Relation을 정의하도록 구성하였다.

최종 Ontology는 `common.ttl`, `etf_kr.ttl`, `etf_gl.ttl`, `fund_pub.ttl`, `bond_kr.ttl`의 다섯 개 Turtle Module로 구성된다. `common.ttl`에는 금융상품, 기관 역할, 지역, 자산유형, 위험등급, 통화, 식별자, 지수 등 상품군 전반에서 공통으로 사용되는 개념을 정의하고, 나머지 네 Module에서는 각 상품군에서 필요한 Class와 Property를 추가하였다.

따라서 다섯 파일은 서로 독립적인 Ontology가 아니라, 하나의 금융상품 의미 체계를 **공통 Module**과 **상품군별 확장 Module**로 분리한 구조로 동작한다.

Source Data → Canonical Representation → Ontology Semantics → Retrieval / Agent Grounding

4.1 Canonical Data Model

Canonical Data Model은 서로 다른 원천 데이터의 Column을 하나의 물리적 Schema로 단순 통합하기 위한 구조가 아니다. 원천 데이터에 포함된 금융상품, 세부 판매 단위, 수치, 관계 및 분류 정보를 Agent가 일관된 의미로 탐색할 수 있도록 공통 identity와 사실 표현으로 정규화하는 실행 계층이다.

본 시스템에서는 이를 PostgreSQL의 별도 `canonical_v2` Schema로 구현하고, Schema 변경은 Alembic Migration으로 관리한다. 따라서 Canonical Data Model은 개념적인 설계에 그치지 않고, 데이터 적재, Entity Resolution, 관계 생성, 수치 조회 및 Evidence 추적에 실제로 사용되는 Runtime Data Model로 동작한다.

4.1.1 원천 데이터 구성

본 프로젝트에서 사용하는 데이터는 국내채권, 국내 ETP, 해외 ETP, 공모펀드의 네 데이터셋으로 구성된다. 각 데이터셋은 서로 다른 Schema뿐 아니라 서로 다른 데이터 grain을 갖는다.

표 3: 금융상품 원천 데이터 구성

데이터셋	대상	행 수	열 수	주요 데이터 단위
PRBD01N001	국내채권	21,882	58	채권 상품 및 판매 관련 행
PREF01N001	국내 ETF · ETN	1,780	98	상장상품
PREF02N001	해외 ETF · ETN	6,037	49	상장상품
PRFD01N001	공모펀드	23,676	75	펀드 클래스
합계	4개 데이터셋	53,375	280	–

원천 데이터의 한 행은 항상 하나의 독립적인 금융상품을 의미하지 않는다.

공모펀드 데이터는 클래스 단위로 제공되므로, 여러 행이 하나의 부모 Fund에 속할 수 있다. 이에 따라 펀드 자체를 나타내는 Fund와 클래스별 판매 단위를 나타내는 FundShareClass를 구분하였다.

국내채권 데이터에서도 동일한 상품번호 `pd_no`가 장내 · 장외 시장 구분 및 장외 판매조건 `info_seq`에 따라 복수 행으로 나타날 수 있다. 특히 장외 데이터에는 동일 채권에 대해 판매가격, 매수 수익률, 매수가능수량 등이 다른 복수의 판매조건이 존재한다. 따라서 채권 상품 자체를 나타내는 Bond와 개별 판매조건을 나타내는 SaleLot을 분리하였다.

이와 같이 본 시스템은 원천 행 수를 금융상품 수로 직접 해석하지 않고, 각 행이 증명하는 실제 업무 단위와 데이터 grain을 기준으로 Canonical Entity와 Fact를 구성한다.

4.1.2 Canonical Model의 구성 영역

canonical_v2 Schema는 원천 이력, 공통 Entity, 금융상품, 세부 판매 단위, 식별자, 사실 및 Evidence를 다음과 같이 구분하여 관리한다.

표 4: Canonical Data Model의 주요 영역

모델 영역	주요 테이블	책임
원천, 실행 이력	source_datasets, dataset_snapshots, ingestion_runs, source_records	출처, 데이터 세대, Checksum, 원문 Record 및 적재 상태를 관리한다.
개체, 상품	canonical_entities, financial_products, bonds, exchange_traded_products, funds	공통 Entity identity와 상품 유형을 관리하고, 상품군별 상세 정보를 공통 identity에 연결한다.
세부 단위	fund_share_classes, sale_lots	펀드 클래스와 채권 판매조건을 부모 금융상품과 구분된 grain 으로 관리한다.
식별, 연결	entity_identifiers, entity_aliases, source_record_entities	식별자 Scheme, Alias 및 원천 Record와 Canonical Entity의 연결을 관리한다.
의미 사실	canonical_facts Scalar, Relation, Classification 테이블	해결된 Entity에 귀속되는 값, 관계 및 분류를 공통 Semantic Key를 기준으로 관리한다.
지표, Evidence	metric_observations, source_field_assertions, fact_evidence_links	수치의 의미와 기준시점을 관리하고, Canonical Fact를 원천 Field까지 추적한다.

4.1.3 공통 Entity identity

canonical_entities는 상품뿐 아니라 기관, 지수, 증권 등 시스템이 식별한 모든 Canonical Entity의 공통 identity를 제공한다. 각 Entity는 개체 유형, 대표 이름, 이름의 권위 상태, Resolution 상태 및 질의 사용 가능 여부를 갖는다.

financial_products는 이 가운데 금융상품에 해당하는 Entity를 Bond, ETF, ETN, Fund로 구분한다. 상품군별 상세 테이블은 동일한 Canonical Entity를 참조하므로, 여러 저장소와 데이터 셋에서도 하나의 공통 Identifier를 기준으로 동일 상품을 연결할 수 있다.

개념적으로는 다음과 같은 구조를 갖는다.

Canonical Entity → Financial Product → Bond / ETF / ETN / Fund

FundShareClass와 SaleLot은 금융상품 본체와 다른 grain을 갖는 별도 Entity이다. 두 Entity는 각각 부모 Fund와 Bond를 참조하지만, 그 자체를 별도의 금융상품 수로 집계하지 않는다.

4.1.4 Canonical 금융상품 집계

원천 식별자와 상품군별 변환 규칙을 적용하여 확정된 Canonical FinancialProduct는 총 35,180개이다. 상품 유형별 구성은 다음과 같다.

표 5: Canonical FinancialProduct 구성

상품 유형	Canonical Entity 수	비고
Bond	20,497	채권 상품 본체
ETF	7,206	국내、해외 ETF
ETN	610	국내、해외 ETN
Fund	6,867	권위 있는 부모 identity가 확인된 Fund
합계	35,180	FinancialProduct 집계

이와 별도로 FundShareClass 16,574개와 SaleLot 634개를 관리한다. 이들은 상품 본체가 아닌 클래스 및 판매조건 단위이므로 35,180개의 FinancialProduct 집계에는 포함하지 않는다.

또한 공모펀드 원천 가운데 권위 있는 부모 Fund identity를 확정할 수 없는 7,102개 행은 임의의 Fund로 묶거나 새로운 부모 상품으로 생성하지 않는다. 이는 원천 데이터의 행을 가능한 한 많이 상품으로 변환하는 것보다, 검증된 identity만 Canonical Product로 승격한다는 본 시스템의 원칙을 반영한다.

위 집계는 원천 데이터와 변환 규칙을 이용한 재계산 결과이며, 운영 데이터베이스의 단순 Row Count와는 구분된다.

4.1.5 Canonical Fact와 수치 관측

canonical_facts는 확정된 Entity에 귀속되는 의미 사실을 관리한다. 각 Fact는 최소한 다음 정보를 포함한다.

- 사실의 주체가 되는 Canonical Entity
- 해당 사실이 유효한 Dataset Snapshot
- Scalar, Relation, Classification 등의 사실 유형
- 의미를 식별하는 Semantic Key
- Resolution 및 Validation 상태
- 필요한 경우 유효기간

Scalar 값은 TEXT, NUMERIC, DATE, BOOLEAN 가운데 의미에 적합한 하나의 Type으로 저장한다. 수치 값은 문자열로 저장하지 않고 정확한 비교와 Ranking이 가능하도록 NUMERIC(38, 15) 형식을 사용한다.

관계 사실과 분류 사실은 전용 표현을 이용하여 의미를 구체화한다. 예를 들어 상품과 운용기관의 관계, ETF와 구성종목의 관계, 상품과 자산유형의 관계는 단순 문자열 값이 아니라 Entity 또는 통제된 분류 개념을 가리키는 Canonical Fact로 관리한다.

AUM, 가격, NAV, 수익률과 같이 시점 또는 기간에 따라 달라지는 수치는 metric_observations에서 관리한다. 따라서 동일한 상품의 동일한 Metric이라도 Snapshot, 관측 시점, 기간 또는 의미가 다르면 서로 다른 Observation으로 구분할 수 있다.

이는 다음과 같은 비교 오류를 방지한다.

- 서로 다른 기준일의 AUM을 동일한 값처럼 비교하는 경우
- 1년 수익률과 3년 수익률을 동일한 Metric으로 처리하는 경우
- 가격과 NAV를 구분하지 않고 Ranking하는 경우
- 단위 또는 통화가 다른 수치를 직접 비교하는 경우

4.1.6 원문 Evidence 추적

Canonical Model은 정규화된 값만 보존하지 않고, 해당 사실이 어느 원천 데이터의 어떤 Record와 Field에서 유래했는지를 함께 관리한다.

`source_field_assertions`는 원천 Record의 Field 단위 값을 보존하고, `fact_evidence_links`는 해당 Assertion을 Canonical Fact와 연결한다.

따라서 하나의 Canonical Fact는 다음과 같은 Provenance Chain을 갖는다.

**Source Dataset → Snapshot → Source Record → Field Assertion → Evidence Link
→ Canonical Fact**

동일한 사실을 복수의 원천 Record가 증명하는 경우에도 Canonical Fact를 중복 생성하지 않는다. 대신 하나의 Fact에 복수의 Evidence Link를 연결하여, 사실의 identity와 근거의 수를 분리한다.

4.1.7 불확실성과 충돌의 명시적 보존

Canonicalization의 목적은 원천 데이터의 모든 공백을 임의로 채우는 것이 아니다. 본 시스템은 이름이나 identity를 확정할 근거가 부족하거나, 서로 다른 원천 값이 충돌하는 경우를 별도의 상태로 보존한다.

예를 들어 부모 Fund의 권위 있는 이름을 확인할 수 없는 경우에는 `NO_AUTHORITATIVE_FAMILY_NAME` 상태를 기록하고, 대표 이름인 `preferred_name`을 NULL로 유지한다. 클래스명은 `FundShareClass`의 이름 또는 검색용 Alias로 사용할 수 있지만, 이를 공식 부모 Fund의 이름으로 승격하지 않는다.

마찬가지로 다음 상태를 서로 구분한다.

- 원천 값은 존재하지만 Entity identity가 확정되지 않은 경우
- Entity는 확정되었지만 서로 다른 값이 충돌하는 경우
- 부모 Entity를 결정할 수 없는 경우
- 사실은 존재하지만 질의에 사용하기 위한 검증을 통과하지 못한 경우

이를 통해 현재 해결된 지식과 향후 외부 데이터로 보완해야 할 지식을 명확히 구분할 수 있다. 불확실한 값을 임의로 확정하지 않는 이러한 구조는 최종 질의응답 단계의 Fail-Closed 정책과도 연결된다.

4.1.8 Ontology와의 역할 구분

Ontology는 금융상품의 Class, Property, Relation과 의미 제약을 정의하고, Canonical Data Model은 그 의미에 대응하는 Entity, Fact, Observation과 근거를 실제 저장·조회하는 구조를 제공한다. 따라서 모든 관측값과 원문 Assertion을 TTL이나 Neo4j에 중복 저장하지 않는다.

두 계층은 Canonical Concept와 Canonical Field를 통해 연결된다. Concept는 상품 유형·지역·자산군의 공통 의미를, Field는 질의 조건과 조회 항목의 공통 의미를 표현한다. 구체적인 대응 규칙과 실제 조회 경로는 4.7절에서 설명한다.

이 구조에서는 원천 Schema나 물리 저장 위치가 달라져도 상위 Agent가 공통 의미를 유지할 수 있다. 새 Source를 수용할 때에는 해당 Source의 변환 규칙과 Mapping을 추가하고, 필요한 식별·지표·관계 제약을 검증한 뒤 실행 범위에 포함한다.

4.2 금융상품 Ontology 설계

Ontology는 금융상품의 실제 데이터 값을 별도로 저장하기 위한 구조가 아니라, 금융상품이 어떤 Class에 속하는지, Class 사이에는 어떠한 계층이 존재하는지, 그리고 Entity 사이에 어떠한 Relation

을 허용하는지를 정의하는 Semantic Schema이다.

공통 Namespace는 <https://miraeasset.com/ontology/financial-product#>이며, 공통 의미와 상품군별 의미를 다섯 개 Turtle Module로 분리하였다. 현재 활성 Module을 병합한 그래프는 1,276개의 RDF Triple, 63개의 명시적 OWL Class, 55개의 Object Property, 36개의 Datatype Property와 12개의 SHACL NodeShape를 포함한다. 이 수치는 Ontology 파일에 선언된 구조의 집계이며, 운영 데이터베이스의 Entity·Relation 수나 추론 후 생성되는 Class 수를 의미하지 않는다.

4.2.1 Module과 Class 구조

표 6: Ontology Module 구성

Module	주요 내용
common.ttl	금융상품, 기관, 증권, 식별자, 관측, 출처, 증거 및 공통 분류와 Relation을 정의한다.
etf_kr.ttl	국내 ETF·ETN Class와 레버리지 배수, 분배주기 등 국내 ETP 특화 Property를 정의한다.
etf_gl.ttl	해외 ETF·ETN Class와 투자전략, 복제방식 등 해외 ETP 특화 Relation·Property를 정의한다.
fund_pub.ttl	Fund와 FundShareClass, 공모 여부, 환매 유형 및 투자자 자격 등 펀드 특화 의미를 정의한다.
bond_kr.ttl	국내채권 유형, SaleLot, 신용등급, 발행일, 만기일 및 채권 특화 Property를 정의한다.

Ontology의 중심에는 FinancialProduct를 두고, 상품 구조에 따라 Bond, ExchangeTradedProduct, FundProduct 및 관련 하위 개념을 구분하였다. ETF는 상장지수상품이면서 펀드 성격을 갖고, ETN은 상장지수상품이면서 채무증권 성격을 갖는다는 복수의 의미 축을 표현한다.

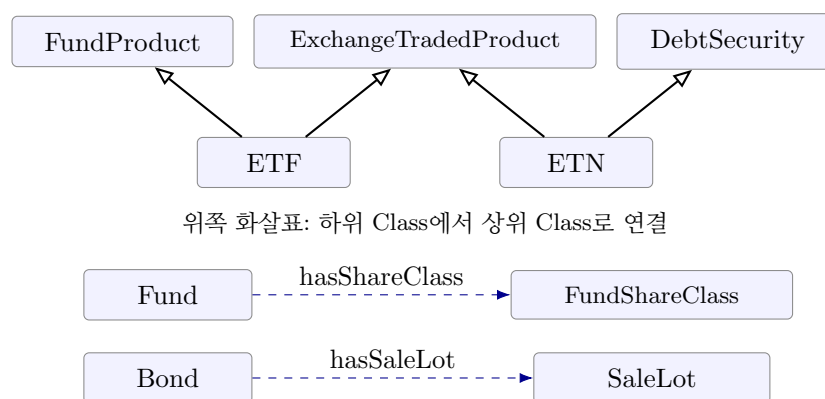


그림 6: 상품의 다중 상속과 상품 본체·판매 단위의 관계 구분

ETF와 ETN은 동일한 ETP 데이터셋에서 제공되지만 상품의 구조가 서로 다르므로 별도의 Class로 정의하였다. 또한 owl:disjointWith를 이용하여 하나의 Entity가 ETF와 ETN으로 동시에 분류되는 것을 제한하였다.

Fund와 FundShareClass, Bond와 SaleLot은 상품 본체와 세부 증거 단위가 서로 다른 identity를 갖도록 분리하였다. 또한 “공모펀드”는 별개의 상품 본체 유형으로 단순화하지 않고, Fund와 공모

여부 분류의 조합으로 해석한다. 이와 같은 구조는 상품 유형, 판매 단위, 공모 여부를 서로 다른 의미 축으로 보존한다.

4.2.2 제약과 실행 범위

Ontology는 Class hierarchy와 `owl:disjointWith`, Object Property의 Domain, Range, Datatype Range 및 SHACL NodeShape를 통해 의미적으로 허용되는 구조를 표현한다. 예를 들어 ETF와 ETN의 동시 분류를 제한하고, 상품 본체와 FundShareClass, SaleLot의 grain을 구분하며, 관계의 주체와 대상이 허용된 Entity Type인지 검사할 기준을 제공한다.

다만 Ontology에 개념이나 Relation이 선언되어 있다는 사실만으로 해당 기능이 현재 질의에서 모두 실행 가능하다는 뜻은 아니다. Runtime Mapping에는 Concept, Field 및 Relation의 의미 대응과 활성 상태가 별도로 정의되고, RDB Field Registry, Graph Projection Allowlist 및 Metric Capability가 실제 조회 가능 범위를 결정한다. 또한 SHACL과 OWL 선언, 오프라인 검증 도구, 온라인 서비스가 강제하는 제약의 범위도 구분한다.

가격, NAV, AUM, 수익률과 같은 시점형 수치는 Ontology에서 의미를 정의하되, 실제 관측값은 Canonical DB의 Observation으로 관리한다. Graph 추출기는 관계 탐색에 필요한 Entity와 Relation을 투영하며, 모든 Metric Observation과 원문 Evidence를 Neo4j에 중복 저장하지 않는다. 즉 Ontology의 표현 가능 범위와 각 저장소에 실제로 투영하는 범위를 분리한 것이 설계의 핵심이다.

표 7: Ontology 설계가 구분하는 대표 의미 질문

대표 질문	필요한 의미 구분
미국에 투자하는 주식형 ETF는 무엇인가?	상품 유형, 투자노출 지역, 자산군을 서로 다른 조건으로 해석한다.
특정 Fund에 속한 판매 클래스는 무엇인가?	Fund와 FundShareClass의 identity 및 grain을 구분한다.
현재 판매 가능한 채권의 판매조건은 무엇인가?	Bond와 SaleLot, Snapshot 기준의 판매 상태를 구분한다.
특정 기업이 발행한 증권을 편입한 ETF는 무엇인가?	ETF-Security 보유 관계와 Security-Organization 발행 관계를 순회한다.
어떤 상품의 1년 수익률이 더 높은가?	관측기간, 기준일, 단위, Source 및 비교 Capability를 함께 확인한다.

4.3 주요 Entity

본 시스템의 지식 모델은 원천 데이터의 각 Column을 그대로 Entity로 변환하지 않는다. 금융 질의에서 독립적인 탐색 대상이 되거나, 여러 상품이 공통으로 참조하거나, 별도의 식별자와 생명주기를 갖는 대상을 중심으로 Canonical Entity를 구성한다.

반면 상품명과 같이 개별 상품에 직접 귀속되는 값이나 검색과 필터링에 사용되는 분류 값은 Datatype Property 또는 Canonical Field로 표현한다. 가격, 순자산, 수익률처럼 시점에 따라 변하는 값은 기준일과 단위를 함께 보존할 수 있는 관측 데이터로 관리한다.

본 절에서 사용하는 Entity는 OWL/Turtle에 정의된 Domain Ontology의 Class와 실제 RDB 및 Knowledge Graph에서 사용하는 Canonical Runtime Entity를 함께 포함한다. Domain Ontology가 금융 개념의 의미와 허용 관계를 정의한다면, Canonical Runtime Model은 원천 데이터의 실제 grain과 질의 실행 단위를 표현한다.

표 8: 주요 Entity와 의미적 역할

구분	대표 Entity	의미와 실제 구분
Financial Product	FinancialProduct, Bond, ETF, ETN, Fund	금융상품 본체의 identity와 상품군을 표현한다.
Product Grain	FundShareClass, SaleLot	Fund 및 Bond 본체와 구별되는 판매 클래스와 채권 판매 단위를 표현한다.
Organization	Organization	운용, 발행, 수탁 역할을 수행하는 기관을 표현한다. 역할은 Relation으로 구분하고, 이름 기반 identity는 Source 범위를 유지한다.
Index	Index	ETP의 기초, 추종지수와 Fund의 벤치마크 관계가 가리키는 대상이다.
Portfolio Security	Security	ETF의 편입 대상 증권을 표현한다. 현재 확장 모델은 주식 증권을 중심으로 발행기관과 연결한다.
Country / Currency	Country, Currency	상품의 상장국가와 표시통화에 대응하는 표준화된 참조 Entity이다.
Classification	AssetClass, ExposureRegion, RiskGrade, PublicOfferingStatus 등	자산군, 투자노출 지역, 위험등급, 공모 여부 등 검색과 필터링에 사용되는 통제된 금융 개념을 표현한다.
Identifier / Provenance	ProductIdentifier, Dataset, Snapshot, SourceRecord, FieldAssertion	Canonical identity 결정과 원천 데이터의 Record, Field 수준 추적에 사용된다.

기관과 지수처럼 권위 식별자가 충분하지 않은 Entity는 정규화된 이름과 Entity Type, Dataset 범위를 결합하여 identity를 구성한다. 다른 Source에서 이름이 같다는 이유만으로 동일한 기관이나 지수로 전역 병합하지 않으며, 교차 Source 연결에는 별도의 권위 식별 근거를 요구한다. 이는 일부 중복 후보를 허용하더라도 잘못된 동일성 확정을 줄이기 위한 선택이다.

또한 Ontology에 출처, 증거 개념을 정의했다는 사실이 원문 Payload와 모든 Assertion을 Neo4j에 적재한다는 뜻은 아니다. 실제 원문과 Field Assertion, Hash는 PostgreSQL과 Artifact에 보존하고, 조회 결과에는 해당 Evidence를 역추적할 수 있는 Identifier를 포함한다.

4.4 주요 Relation과 Graph Projection

Relation은 Entity 사이에 어떠한 금융적 의미의 연결이 존재하는지를 정의한다. 본 시스템에서는 자연어 질의를 Ontology Property에 Grounding한 뒤, 실제 관계 탐색이 필요한 경우 해당 의미를 Knowledge Graph의 Edge Type으로 Projection한다.

이때 Ontology Property와 Graph Edge는 완전히 동일한 목록을 의미하지 않는다. `hasAssetType`, `hasRiskGrade`와 같이 정형 필터링에 주로 사용되는 의미는 Canonical Field와 RDB를 통해 실행될 수 있다. 반면 구성종목, 발행기관, 운용기관처럼 Entity 사이의 탐색 경로가 중요한 정보는 Knowledge Graph의 Edge로 구성한다.

현재 Runtime Knowledge Model에서는 다음의 주요 의미 관계를 Graph Edge로 사용한다.

표 9: 주요 의미 Relation과 Knowledge Graph Edge

의미 Relation Graph Edge	주체 → 대상	생성 조건 및 의미
managedBy MANAGED_BY	ETF、Fund → Organization	ETF 운용사 또는 Fund 운용기관에 관한 원천 근거를 바탕으로 생성한다.
issuedBy ISSUED_BY	Bond、ETN → Organization	Bond의 발행기관을 연결한다. ETN은 발행기관을 명확히 확인할 수 있는 근거가 있는 경우에만 연결한다.
hasTrustee HAS_TRUSTEE	Fund → Organization	수탁기관 Field와 Fund 단위의 identity 충돌 여부를 검사한 뒤 연결한다.
hasUnderlyingIndex HAS_UNDERLYING_INDEX	ETF、ETN → Index	원천 데이터에 제공된 기초지수명을 정규화된 Index Entity와 연결한다.
tracksIndex TRACKS_INDEX	명시적 추종 ETP → Index	해외 원천 데이터의 추종 여부 등 추가적인 근거가 확인된 경우에 사용한다.
hasBenchmark HAS_BENCHMARK	Fund → Index	펀드의 벤치마크 원천과 다른 Index Relation의 충돌 여부를 검사한다.
hasShareClass HAS_SHARE_CLASS	Fund → FundShareClass	검증된 부모 Fund identity를 기준으로 펀드와 판매 클래스를 연결한다.
hasSaleLot HAS_SALE_LOT	Bond → SaleLot	실제 판매가격, 수익률, 수량 등 판매조건에 대한 원천 Evidence가 존재할 때 생성한다.
denominatedIn DENOMINATED_IN	FinancialProduct → Currency	상품의 표시통화를 연결한다. 거래통화가 별도로 존재하는 경우에는 구분하여 관리한다.
listedInCountry LISTED_IN_COUNTRY	ETF、ETN → Country	ETP가 상장된 국가를 표현하며, 상품의 투자지역과는 구분한다.
holds HOLDS	ETF → Security	검증된 외부 보유내역과 ETF、Security 양측의 Canonical identity가 확인된 경우에만 생성한다.
securityIssuedBy SECURITY_ISSUED_BY	Security → Organization	별도의 증권 권위자료를 바탕으로 Security와 발행기관을 연결한다.

Ontology에 선언된 Relation, Canonical Fact로 생성할 수 있는 Relation, Neo4j에 투영 가능한 Edge와 자연어 질의에서 실제 호출할 수 있는 Relation은 서로 같은 범위가 아니다. 현재 Ontology에는 55개의 Object Property가 선언되어 있지만, v2 Graph Projection Allowlist는 21개 Relation을, Query Graph Mapping은 그중 20개 Relation을 다룬다. 예를 들어 가입상태 관계는 Graph에 투영될 수 있으나 현재 자연어 Query Mapping에는 포함되지 않는다. 표 9는 이 가운데 금융상품 탐색에서 핵심적으로 사용하는 Relation을 정리한 것이다.

이 구분을 통해 Ontology에 표현할 수 있다는 사실과 현재 데이터로 Fact가 생성되었다는 사실, 그리고 Agent가 해당 Relation을 실제 질의에 사용할 수 있다는 사실을 동일하게 취급하지 않는다.

Graph Edge는 단순한 문자열 유사성만으로 생성하지 않는다. Relation마다 주체와 대상의 Entity Type, Canonical Identifier, 원천 Evidence 및 생성 조건을 확인한다.

예를 들어 HOLDS는 ETF와 Security 양측의 identity가 확인되고, 검증된 구성종목 데이터가 존재할 때만 생성한다. 또한 SECURITY_ISSUED_BY는 Security와 Organization 사이의 발행 관계를 별도의 권위자료로 확인할 수 있을 때만 생성한다.

따라서 다음과 같은 관계 경로를 탐색할 수 있다.

$$\text{ETF} \xrightarrow{\text{HOLDS}} \text{Security} \xrightarrow{\text{SECURITY_ISSUED_BY}} \text{Organization}$$

Graph는 역방향으로도 Traversal할 수 있으므로, 특정 Organization이 발행한 Security를 식별한 뒤 해당 Security를 편입한 ETF 후보를 탐색할 수 있다.

예를 들어

“삼성전자가 발행한 증권을 편입한 ETF를 알려줘.”

라는 질의는 개념적으로 다음과 같이 처리된다.

Organization Resolution → Security Issuer Traversal → ETF Holdings Traversal → Candidate ETF

여기에 “순자산이 큰 3개”와 같은 조건이 추가되면, Graph에서 식별한 ETF 후보의 Canonical Identifier를 RDB Retrieval에 전달하고, 해당 Source · 통화 · 기준시점 정책에 따라 비교 가능한 AUM을 조회하여 Ranking을 수행한다.

Graph Candidate Retrieval → RDB Metric Retrieval → Ranking → Evidence Validation

다만 현재 Relation 범위만으로는 기업과 기업 사이의 자회사 관계를 직접 탐색할 수 없다. “특정 기업의 자회사가 발행한 증권을 편입한 ETF”와 같은 Multi-hop 질의를 지원하려면, 권위 있는 외부 기업관계 데이터에 근거한 다음 Relation이 추가로 필요하다.

$$\text{Organization} \xrightarrow{\text{SUBSIDIARY_OF}} \text{Organization}$$

해당 Relation이 구현되고 검증되기 전에는 Agent가 기업명 유사성이나 LLM의 내부 지식을 이용하여 자회사 관계를 임의로 생성하지 않는다. 현재 지원하지 않는 관계가 질의에 포함된 경우에는 이를 생략하여 부분적인 답을 반환하는 대신 Unsupported Relation으로 처리한다.

이와 같이 Ontology와 Graph Relation Registry는 자연어에 등장한 관계를 허용된 Entity Type과 검증된 실행 경로에 연결한다.

Natural Language Relation → Ontology Property → Domain / Range Validation → Graph Edge Type → Bound Traversal

이를 통해 Agent가 원천 Column명이나 문자열 유사성만으로 관계를 추정하지 않고, 명시적인 금융적 의미와 Source Evidence를 기준으로 관계 탐색 경로를 결정하도록 하였다.

4.5 금융 지표 및 속성 모델링

금융상품의 모든 정보를 별도의 Entity와 Relation으로 표현할 필요는 없다. 다른 Entity와의 연결 자체가 의미를 갖는 정보는 Object Property로 표현하고, 상품명이나 만기일처럼 개별 상품에 직접 귀속되는 비교적 안정적인 값은 Datatype Property 또는 Canonical Scalar Fact로 관리한다.

반면 가격, NAV, AUM, 수익률과 같이 시점이나 기간에 따라 달라지는 금융 지표는 단순한 숫자 하나만으로 의미를 충분히 표현할 수 없다. 동일한 수치라도 지표의 종류, 단위, 통화, 기준일, 산출 기간 및 Source가 다르면 서로 직접 비교할 수 없기 때문이다.

본 시스템에서는 이러한 지표의 의미를 Ontology Property와 Canonical Field로 정의하고, 실제 시점별 값은 Canonical Runtime Model의 `metric_definitions`와 `metric_observations`에서 관리한다.

`metric_definitions`는 Canonical Field를 지표 코드 및 허용된 Filter, Comparison, Ranking Capability와 연결한다. `metric_observations`는 원천 표현인 Raw Value와 함께 정규화된 Numeric 또는 Date Value, Unit, Scale Basis, Currency, 관측일인 `observed_on`, Quality 및 Comparability Status를 저장한다.

각 관측값은 Canonical Fact 및 원천 Field Assertion과 연결되므로, 최종 답변에 사용된 수치가 어떤 데이터셋의 어떤 Field에서 유래했는지를 역방향으로 추적할 수 있다.

Ontology Metric Property → Canonical Field → Metric Definition → Metric Observation → Source Evidence

이와 같이 Ontology와 Canonical Runtime Model은 동일한 수치를 중복 저장하지 않고 서로 다른 역할을 담당한다. Ontology는 지표가 어떠한 금융적 의미를 갖는지를 정의하고, Canonical Runtime Model은 실제 값과 기준일, 단위, 통화 및 품질 상태를 관리한다.

현재 시스템에서 사용하는 주요 금융 지표와 실행 범위는 다음과 같다.

표 10: 주요 금융 지표, 속성과 실행 범위

지표, 속성	현재 원천과 단위	실행 의미 및 제한
ETP 순자산	<code>du_last_aum</code> 국내 KRW, 해외 계약 USD	허용된 Source와 통화 범위에서 Ranking을 수행하며, 자동 환산은 적용하지 않는다.
ETP 보수율	<code>cu_charge_rt</code>	원천 비율값의 Scale이 검증되지 않아 비교용 Filter와 Ranking을 활성화하지 않는다.
국내 1년 수익률	<code>du_er_1y</code> 관측일: <code>du_upt_dt</code>	원천의 퍼센트 표현을 보존하며, 승인된 국내 ETP 모집단 안에서만 비교와 Ranking을 수행한다.
해외 기본 1년 수익률	PREF02에 해당 Field 없음	<code>du_er_1d</code> 등을 변환산하거나 1년 수익률의 대체 Field로 사용하지 않는다.
iShares 1년 수익률	<code>oneYearAnnualized.</code> <code>navSourced</code>	승인된 NAV Total Return과 월말 기준 범위에서만 사용한다.
펀드 1년 수익률	<code>fd_yr1_ern_r</code> <code>fd_price_bas_dt</code>	FundShareClass 단위의 관측값으로 관리하며, 부모 Fund의 성과로 임의 승격하지 않는다.
채권 신용등급	<code>crd_grd</code> 등급정보 기준일	코드에 정의된 15개 등급의 순서척도를 이용한 Filter만 수행한다.
판매 가능 채권	기준 Snapshot의 Lifecycle Fact	원천 데이터 기준의 판매 가능 상태이며, 실시간 주문 가능 여부나 재고를 의미하지 않는다.

금융 지표의 기준일은 데이터셋의 대표 Snapshot 날짜로 일괄 대체하지 않고, 각 지표에 대응

하는 원천 Field를 사용한다. 예를 들어 해외 종가의 기준일은 `du_clpr_base_dt`이고, 국내 ETP 지표에는 `du_upt_dt`가 사용된다. 외부 데이터 역시 별도의 `effective_date`를 갖는다.

따라서 모든 관측일을 하나의 Snapshot 날짜로 일괄 치환하지 않는다. 그렇게 처리하면 실제로 값이 관측되거나 유효했던 시점이 사라져, 서로 다른 기준일의 금융 지표가 동일한 시점의 값처럼 비교될 수 있기 때문이다.

결측값과 숫자 0 역시 구분한다. NULL은 값이 존재하지 않거나 해석할 수 없는 상태를 의미하므로 임의로 0으로 변환하지 않는다. 원천 데이터에 실제로 기록된 0은 `SOURCE_ZERO` 상태와 함께 보존한다. 다만 거래량처럼 0 자체가 유효한 관측값인 경우에는 업무적 의미에 따라 정상적인 값으로 사용할 수 있다.

즉 숫자를 Numeric Type으로 변환할 수 있다는 사실과 해당 값을 실제 비교나 Ranking에 사용할 수 있다는 사실은 서로 다르다. 비교 가능한 지표가 되기 위해서는 유효한 값뿐 아니라 Unit, Scale, Currency, 관측일 및 Metric Definition의 호환성이 확인되어야 한다.

지표의 연산 가능 여부는 `MetricCapabilityRegistry`가 Source와 요청 모집단을 함께 고려하여 결정한다. 예를 들어 국내 ETP의 1년 수익률과 iShares의 1년 수익률은 모두 퍼센트로 표현되지만, NAV와 시장가격의 기준, 분배금 반영 방식 및 산출 범위가 충분히 동일하다고 검증되지 않았다. 따라서 두 Source의 수익률을 하나의 모집단으로 통합하여 Ranking하지 않는다.

이는 단위만 같다는 이유로 서로 다른 산출 기준을 가진 금융 지표를 직접 비교하는 방식과 구분된다. 본 시스템은 숫자의 형식뿐 아니라 Metric의 정의와 비교 가능성을 함께 검증한 뒤 Filter와 Ranking을 수행한다.

의미 Field가 정의되어 있다는 사실과 실제 Query Capability가 활성화되어 있다는 사실도 구분한다. 일반 수익률, 채권 Yield 및 만기 관련 Field는 Runtime Mapping의 확장 후보로 등록되어 있지만, 현재 검증을 거쳐 활성화된 특정 1년 수익률 Field와 동일한 조회·비교 Capability를 제공하는 것은 아니다.

즉 원천 데이터에 값이 존재하거나 Canonical Model에 저장 위치가 있다는 사실만으로 해당 Field에 대한 모든 자연어 범위 조건과 Ranking을 지원하는 것으로 판단하지 않는다.

위험등급과 신용등급, 표시통화와 거래통화, 판매상태와 상장상태 역시 표현이 비슷하더라도 서로 다른 금융적 의미를 갖는다. 본 시스템은 이러한 값을 서로 다른 Canonical Field와 Semantic Key로 분리하여 Grounding 및 Query Planning 단계에서 구분한다.

따라서 본 프로젝트의 수치 처리 강점은 범용적인 금융 계산을 가능한 한 많이 제공하는 데 있지 않다. 핵심은 특정 연산이 어떤 Source, 모집단, 단위, 통화, 관측 시점 및 산출 기준에서 유효한지를 명시하고, 검증된 범위에서만 Filter, Comparison 및 Ranking을 허용한다는 데 있다.

4.6 식별자 모델 및 Entity Resolution

금융상품은 상품군과 Source에 따라 서로 다른 식별체계를 사용하므로, 모든 식별자를 하나의 `productCode`에 넣지 않는다. 동일한 문자열도 Scheme과 Namespace가 다르면 다른 대상을 의미할 수 있고, 반대로 하나의 상품은 Source 상품번호, ISIN, Ticker 등 복수의 식별자를 가질 수 있기 때문이다.

Ontology에서는 `ProductIdentifier`와 `identifiedBy Relation`으로 식별의 의미를 표현하고, Canonical Runtime Model의 `entity_identifiers`가 실제 Identifier Value, Scheme, Namespace, Source 및 검증 상태를 관리한다. 식별자의 identity는 개념적으로 다음과 같이 결정된다.

$$\text{Identifier Identity} = \text{Scheme} + \text{Namespace} + \text{Normalized Value}$$

상품 본체와 세부 단위의 식별자도 구분한다. 채권의 상품 코드는 Bond identity에 사용하지만, `info_seq`는 동일 Bond에 연결된 SaleLot을 구분한다. 펀드의 Class Code 역시 `FundShareClass`를 식별하는 값이며, 그 자체를 부모 Fund의 식별자로 승격하지 않는다. ETP는 데이터셋 Namespace

와 상품 코드를 함께 사용하고, Security는 거래소 범위를 갖는 Ticker와 ISIN 등 사용 가능한 강한 식별자를 우선한다.

표 11: Entity Resolution에서 식별 정보의 역할

대상	주요 식별 범위	Resolution 원칙
Bond	상품 코드와 Source Namespace	Bond 본체를 식별하며 <code>info_seq</code> 와 혼합하지 않는다.
SaleLot	부모 Bond와 판매조건 식별자	판매조건 Evidence가 있는 세부 단위로 생성한다.
ETF、ETN	Dataset Namespace와 상품 코드	동일 문자열이라도 국내、해외 Source 범위를 구분한다.
FundShareClass	부모 Fund와 Class Code	클래스 identity와 부모 Fund identity를 분리한다.
Security	거래소 Namespace를 갖는 Ticker, ISIN 등	짧은 Ticker나 이름만으로 해외 증권을 확정하지 않는다.
Organization / Index	Entity Type, Dataset 범위, 정규화 이름	권위 식별 근거 없이 다른 Source의 동명 Entity를 전역 병합하지 않는다.

오프라인 Canonicalization에서는 원천 키와 grain을 기준으로 identity를 결정적으로 구성한다. 온라인 CanonicalV2EntityLookup은 Canonical ID, 사용 가능한 Identifier, Preferred Name 및 Alias의 정확 일치 후보를 수집한다. 후보가 하나이면 RESOLVED, 둘 이상이면 AMBIGUOUS, 후보가 없으면 UNRESOLVED로 판정한다. 여기서 Confidence 값은 Resolver 상태에 부여한 값이며 학습된 확률을 의미하지 않는다.

`skos:prefLabel`과 `skos:altLabel`, Runtime의 `entity_aliases`는 자연어 표현으로부터 후보를 찾는 데 사용한다. 예를 들어 “한국”, “국내”, “대한민국”, “Korea”, “KR”은 동일한 지역 개념 후보로 정규화할 수 있다. 그러나 상품명이나 기관명의 Alias가 같다는 이유만으로 두 Entity의 identity를 동일하게 확정하지는 않는다.

User Expression → Label / Alias Candidate → Identifier Verification → Type and Grain Validation → Canonical Entity

Security Resolution에서는 거래소 맥락을 보존한다. 예를 들어 `XNAS:AAPL`처럼 Namespace가 있는 해외 Ticker와 국내 6자리 KRX Ticker를 구분하며, 짧은 영문 Ticker만으로 임의의 해외 증권에 연결하지 않는다. 또한 현재 ISIN 검증은 기본적인 형식 확인 범위이므로, 전체 Checksum 검증이나 발행기관의 실재 확인까지 수행한다고 과장하지 않는다.

식별자가 충돌하거나 복수 후보가 남으면 가장 유사한 Entity를 임의로 선택하지 않는다. 충돌 후보와 미해결 상태를 보존하고, identity가 확정되기 전에는 Relation 생성과 질의 사용을 제한한다. 이러한 보수적 Resolution은 일부 중복 후보를 남길 수 있지만, 서로 다른 금융상품이나 기관을 잘못 병합하는 위험을 줄인다.

4.7 Data-Ontology Mapping

Data-Ontology Mapping은 원천값을 Canonical Fact로 구성하는 오프라인 변환과, 자연어 표현을 Ontology 및 실행 Field에 연결하는 온라인 의미 매핑으로 구분된다. 본 프로젝트는 Source Column을 Ontology Property나 SQL Column에 직접 강하게 결합하지 않고 Canonical Representation을 중간에 둔다.

Source Column → Canonical Representation → Ontology Class / Property → Query Execution

Ontology에는 canonicalConcept, canonicalField, conceptCategory, sourceDataset, allowAmbiguousAlias 등의 Annotation Property가 정의되어 있다.

canonicalConcept는 Class 또는 분류 개념을 공통 Canonical Concept와 연결한다. 예를 들어 ETF는 FinancialProduct.ETF, Korea는 Region.KR, Equity는 AssetType.Equity에 연결된다.

canonicalField는 Relation 또는 Datatype Property를 Agent와 Query Planner가 사용하는 공통 Field와 연결한다. Runtime Mapping에는 Concept 72개, Field 42개, Relation 18개의 명시적인 의미 대응이 있으며, Field는 Active 39개와 Prospective 3개로 구분된다. 이 개수는 곧바로 지원 질의 기능의 개수를 의미하지 않는다. Active Field라도 상품군별 데이터 존재 여부와 Metric Capability에 따라 Filter와 Ranking이 제한될 수 있다.

표 12: Ontology Term과 Canonical Representation의 대표 Mapping

Ontology Term	Canonical Representation	역할
ETF	FinancialProduct.ETF	ETF 상품유형을 공통 Concept로 정규화한다.
Korea	Region.KR	한국, 국내, KR 표현을 하나의 Region으로 통합한다.
managedBy	product.asset_manager	운용사 관계를 공통 Field와 연결한다.
issuedBy	product.issuer	발행사 관계를 공통 Field와 연결한다.
investsInRegion	product.region	투자지역 조건을 공통 Field와 연결한다.
hasAssetType	product.asset_type	자산유형 조건을 공통 Field와 연결한다.
hasRiskGrade	product.risk_grade	위험등급 조건을 공통 Field와 연결한다.
denominatedIn	product.currency	표시통화를 공통 Field와 연결한다.
productName	product.name	상품명 검색 Field를 통합한다.
aum	product.aum	AUM 관련 수치 Field를 연결한다.
expenseRatio	product.expense_ratio	비용 관련 Field를 연결한다.

v2의 실제 데이터 구축은 검토된 Source Field별 명시적인 변환 규칙을 사용한다. ontology/mappings/column_mapping.csv에 정리된 280개 Column Mapping은 원천 Schema 분석과 의미 대응을 위한 자산이지만, 현재 v2 Rebuild가 이 CSV를 읽어 모든 Column을 자동으로 RDF Fact로 변환하는 구조는 아니다. 이 구분을 명시함으로써 문서화된 Mapping과 현재 실행되는 변환 경로를 혼동하지 않는다.

표 13: Source, Ontology, Runtime Mapping의 대표 사례

원천 또는 표현	Canonical 의미	후속 사용
승인된 ETP 자산분류 원문값	AssetClass Concept IRI	Classification Fact와 정형 조건
승인된 투자지역 원문값	ExposureRegion Concept IRI	투자지역 Filter; 상장국가와 구분
원자적인 ETP 기초지수명	hasUnderlyingIndex	Relation Evidence와 Graph Projection
국내 du_er_1y	ONE_YEAR_RETURN Observation	Source 계약을 통과한 모집단의 Ranking
해외 cu_strtegy	전략 원문 Assertion	전략 문서와 Semantic Retrieval
자연어 Field, Relation Alias	Canonical Field, Relation, URI	Grounded Query와 Query Plan 검증

Classification 원문값은 원문, 정규화 값과 해결 후보를 보존한다. 승인된 Exact Alias로 해소되고

동일 대상 안에서 충돌하지 않을 때만 확정된 Classification Fact를 생성한다. 매핑되지 않은 원문을 삭제하지 않으며, 원문 보존, 의미 매핑 성공, Canonical Fact 확정, 특정 연산 지원을 서로 다른 단계로 구분한다.

Region.US, AssetType.Equity와 같은 Runtime Key는 Ontology URI와 물리 DB 사이의 호환 표현이다. 실제 v2 Table 선택과 SQL 생성은 CanonicalV2FieldRegistry와 Compiler가 담당한다. 따라서 의미 대응은 Runtime Mapping이, 현재의 물리 실행은 v2 Field Registry와 조회기가 각각 책임진다.

이러한 분리를 통해 Source Schema나 물리 Table이 변경되더라도 상위 Agent는 동일한 Canonical 의미를 유지할 수 있다. 새 Source를 추가할 때에도 Ontology 전체나 Query Logic을 다시 작성하기보다, 검토된 Source Transformation과 Canonical Mapping을 추가하는 방식으로 확장한다.

4.8 외부 데이터 및 Provenance 관리

금융상품 질의응답에서는 동일한 금융 개념이 여러 Source에서 제공될 수 있으므로, 값 자체뿐 아니라 해당 정보가 어느 Source에서 유래했는지를 구분할 필요가 있다.

Ontology는 Dataset, Record, Assertion 및 Identifier와 같은 출처, 증거 개념의 의미를 제공한다. 실제 원문 Payload, Source Row와 Field, Hash, 수집 시점, Provider 및 변환 Version은 Canonical Data Model과 외부 Artifact 구조에서 보존한다. 따라서 Ontology에 Provenance 개념을 정의하는 것과 모든 원문 데이터를 RDF 또는 Neo4j에 복제하는 것은 구분된다.

Ontology = 정보의 의미와 허용되는 관계

Canonical Evidence = 확정 Fact와 원천 Assertion의 연결

Artifact Provenance = 원문 파일, URL, Hash, 시점 및 Version

기본 원천의 Record · Field 수준 근거 연결은 4.1.6절의 구조를 사용한다. 외부 데이터에는 여기에 Provider, 수집 시점, 유효시점과 원문 Artifact 검증을 추가한다. 서로 다른 원천값이 충돌하면 원문 Assertion과 Conflict 상태를 보존한다.

외부 데이터는 Provider 정보를 수집하는 단계와 Canonical Entity에 연결하는 단계를 분리한다. 외부 Snapshot은 Provider Identifier, URL, Raw Value, Normalized Value, Effective Date, Retrieved At, Trust Tier, Quality Status, Content Hash와 Parser, Crawler Version을 보존한다. 수집기가 이름 유사성이나 생성형 모델의 추측으로 Canonical Entity ID를 직접 만들어 넣지는 않는다.

현재 외부 Artifact에는 검증된 범위의 ETF 보유내역, Security 발행기관 및 일부 1년 수익률 정보가 포함될 수 있다. 각 Artifact는 Manifest에서 상대 경로, Checksum, Effective Date, 호환 Version과 Required 상태를 갖는다. 다만 Manifest의 READY는 해당 Artifact가 등록된 계약을 만족한다는 뜻이며, 모든 외부 자료가 모든 실행 환경에 적재되었거나 모든 관계 질의를 지원한다는 의미는 아니다.

외부 보유내역을 통합할 때에는 READY Manifest, 승인된 Provider, Trust Tier, Quality Status, 평가 Cutoff 이하의 Effective Date 및 원문 Hash를 확인한다. ETF와 Security 양쪽의 identity가 해결된 경우에만 HOLDS Fact를 생성한다. Security의 복수 식별자가 서로 다른 후보를 가리키면 모호성을 유지하고 이름만으로 확정하지 않는다. 현금과 같은 Non-Security Position도 주식 Security로 변환하지 않는다.

보유비중은 정규화된 숫자만 저장하지 않고 Raw Weight, Normalized Value, Weight Unit 및 Scale을 함께 보존한다. NAV 대비 비중과 비현금자산 대비 비중처럼 분모가 다른 값을 동일한 수치로 비교하지 않는다. 의미가 확인되지 않은 원문 비중은 확정된 Metric Observation과 구분하여 관리한다.

시간 통제에서는 파일을 내려받은 시점과 금융 정보가 실제로 유효한 시점을 구분한다. 본 프로젝트의 평가 Cutoff는 2026-08-24이며, 그 이후 수집된 외부 파일도 Effective Date가 Cutoff 이전인지 별도로 확인한다. 또한 “현재 판매 가능”과 같은 표현은 현재 구현에서 선택된 Snapshot의 판매상태를 의미하며, 실시간 주문 가능 여부나 재고 조회를 의미하지 않는다.

종합하면 본 프로젝트의 Ontology는 금융상품의 모든 값을 RDF로 중복 저장하는 Knowledge Base가 아니라, 서로 다른 Source의 개념, Field, Relation과 증거를 연결하여 Entity Resolution, Query Planning, Retrieval 및 Validation이 공통으로 참조하는 **Semantic Contract**로 기능한다.

Natural Language → Ontology Grounding → Canonical Meaning → RDB / Graph / Semantic Retrieval → Evidence Validation → Grounded Answer

이 구조를 통해 Agent는 원천 Column명이나 LLM의 추측에 직접 의존하지 않고, 명시적으로 정의되고 현재 데이터에서 검증 가능한 금융 의미만을 검색과 최종 답변에 사용할 수 있다.

5. Agent 기반 질의 처리

본 시스템의 온라인 질의 처리 과정은 사용자의 자연어 질의를 곧바로 LLM에 전달하여 답변을 생성하는 방식이 아니라, 질의를 먼저 구조화된 의미 표현으로 변환하고, Ontology를 통해 금융 도메인의 개념과 데이터 필드에 Grounding한 뒤, 검증 가능한 QueryPlan을 생성하여 적절한 Knowledge Source에서 Evidence를 수집하는 방식으로 구성된다.

전체 질의 처리 흐름은 다음과 같다.

Question → Query Understanding → Entity Resolution → Ontology Grounding → Query Planning → Federated Retrieval → Evidence Construction → Validation → HyperCLOVA X → Answer

이 과정의 핵심은 자연어 해석, 검색, 검증, 생성의 책임을 분리하는 것이다. LLM이 직접 데이터 베이스를 탐색하거나 임의의 금융 의미를 결정하도록 하지 않고, Ontology와 구조화된 QueryPlan을 중간 표현으로 사용하여 질의의 의미와 실행 결과를 추적 가능하게 유지한다. 이는 기술세션자료에서 제시한 Query Understanding, Ontology Grounding, Federated Query, Evidence 통합 및 검증의 흐름과 대응한다 [기술세션자료, 21-24쪽].

5.1 Query Understanding

Query Understanding 단계의 목적은 사용자의 자연어 질문을 단순 문자열이 아닌 **실행 가능한 금융 질의의 의미 단위**로 분해하는 것이다.

예를 들어 다음과 같은 질의를 고려할 수 있다.

“국내 상장 ETF 중 미국 주식에 투자하는 상품을 순자산이 큰 순서로 3개 알려줘.”

이 질의에는 단순한 상품명 검색이 아니라 상품 유형, 투자 지역, 자산 유형, 수치 조건, 정렬 기준 및 결과 개수와 같은 여러 의미 요소가 포함되어 있다. 따라서 Query Understanding 단계에서는 이를 다음과 같은 구조로 분해한다.

- **Intent:** 상품 검색, 특정 상품 조회, 비교 등 사용자의 목적
- **Target Entity Type:** ETF, Bond, Fund 등 조회 대상 상품군
- **Filter Constraint:** 상품군별로 지원되는 지역, 자산군, 상태 등 조건
- **Relation Constraint:** 운용사, 발행사, 편입 관계 등 관계 조건

- **Sort / Ranking:** 허용된 Metric과 명시적인 정렬 방향
- **Requested Fields:** 사용자가 최종 답변에서 요구한 속성
- **Limit:** 상위 3개와 같은 결과 개수 제한

현재 구현에서는 구조적으로 해석 가능한 표현에 대해 **deterministic rule 기반 분석**을 우선 적용한다. 금융 수치, 비교 연산자, 정렬 조건처럼 해석 기준이 명확한 요소는 LLM의 자유로운 판단에 의존하기보다 규칙 기반으로 구조화함으로써, 동일한 질문에 대해 일관되고 재현 가능한 Query Understanding 결과를 생성하도록 설계하였다.

반면 규칙만으로 충분히 분석되지 않는 자연어 표현에 대해서는 HyperCLOVA X 기반 Semantic Parsing을 보조적으로 활용할 수 있도록 구성하였다. 다만 LLM이 반환한 해석 역시 즉시 실행되는 것이 아니라, 이후 Ontology Grounding과 Planning Validation을 통과해야 한다.

또한 시스템은 사용자의 질의 의도를 임의로 단순화하거나 해석 가능한 일부 조건만 선택적으로 수행하지 않는다. 예를 들어 지원되지 않는 “추천” 의미를 일반 상품 검색으로 축소하거나, 해석할 수 없는 주관적 표현을 임의의 금융 지표에 대응시키는 방식을 허용하지 않는다. 따라서 질의의 핵심 조건 중 일부를 해석할 수 없는 경우에는 해당 조건을 제거한 채 나머지 질의만 수행하기보다, 질의 전체의 의미가 보존되는지를 우선적으로 판단하도록 설계하였다.

5.2 Entity Resolution

자연어 질의에는 동일한 금융 Entity를 가리키는 다양한 표현이 사용될 수 있다. 상품명은 전체 정식 명칭, 단축 명칭, 종목코드 등으로 표현될 수 있으며, 기업과 운용사 역시 약칭이나 서로 다른 표기로 등장할 수 있다.

Entity Resolution 단계에서는 이러한 표현을 시스템 내부에서 사용하는 **Canonical Entity**에 연결한다.

이를 위해 다음 정보를 단계적으로 활용한다.

1. 입력 문자열의 정규화 및 명시적인 식별자 확인
2. Canonical Name 및 Alias 기반 후보 탐색
3. 질의의 Entity Type 및 주변 Context를 이용한 후보 제한
4. Ontology의 Class 및 Relation 정보를 이용한 의미적 호환성 확인

예를 들어 동일한 “채권”이라는 표현도 질의 문맥에 따라 금융상품 유형인 Bond를 의미할 수도 있고, 자산 유형으로서 Bond Asset을 의미할 수도 있다. 따라서 문자열 일치만으로 Entity 또는 Concept을 선택하지 않고, 해당 표현이 사용된 semantic slot과 Ontology Context를 함께 고려한다.

Entity Resolution의 결과는 단순한 문자열 치환이 아니라, 이후 Query Planning과 Retrieval에서 공통으로 사용할 수 있는 Canonical Identifier로 전달된다.

또한 하나의 표현이 여러 Entity로 해석될 수 있으나 충분한 근거로 하나를 선택할 수 없는 경우에는 임의로 후보를 결정하지 않는다. 이러한 ambiguity는 후속 Validation 단계까지 명시적으로 보존하거나, 질의 자체를 안전하게 처리할 수 없는 경우 답변 제한의 근거로 사용한다.

5.3 Ontology Grounding

Ontology Grounding은 본 시스템의 핵심 단계로, Query Understanding에서 얻은 자연어 수준의 의미를 금융상품 Ontology가 정의하는 **Concept, Relation, Field, Constraint**에 연결한다.

기술세션자료에서는 Ontology를 단순한 용어 사전이 아니라 개념, 관계 및 규칙을 정의하는 Semantic Layer로 설명하며, LLM이 언어를 이해한다면 Ontology는 데이터가 가지는 의미를 이해하는 계층으로 제시한다 [기술세션자료, 11쪽].

본 시스템에서도 Ontology는 단순히 프롬프트에 제공되는 설명 자료가 아니라 Agent의 runtime에서 직접 사용된다. 구체적으로 다음 작업을 수행한다.

- 자연어 표현을 Canonical Concept으로 변환
- 자연어 속성 표현을 Canonical Field에 연결
- Relation의 표기와 Alias를 Ontology Property로 정규화
- Relation의 Domain / Range 호환성 확인
- 질의에서 사용할 수 있는 Field 및 Relation의 의미적 유효성 확인

특히 Ontology의 의미 표현과 실제 데이터베이스의 물리적 Column을 직접 결합하지 않고,

Natural Language → Ontology Concept / Property → Canonical Field → Physical Storage Field

의 단계로 분리하였다.

예를 들어 사용자의 “순자산”이라는 표현은 Ontology의 해당 금융 지표 의미에 Grounding되고, 이후 Canonical Field를 거쳐 실제 RDB에서 사용하는 물리적 Column에 연결된다.

이와 같은 간접 Mapping 구조를 통해 Ontology가 데이터베이스 구현 세부사항에 종속되는 것을 줄이고, 반대로 데이터베이스의 Column 이름만으로 자연어의 금융 의미를 추론하는 문제를 방지한다.

기술세션의 실험에서도 Ontology를 단순 프롬프트 설명으로 제공하는 것보다 runtime의 식별 과정에 활용했을 때 더 큰 성능 개선이 나타났으며, 이를 Ontology가 “설명”이 아닌 “런타임 통제 계층”으로 사용되어야 한다는 결과로 제시하였다 [기술세션자료, 23쪽].

5.4 Query Planning

Ontology Grounding이 “질문의 각 표현이 무엇을 의미하는가”를 결정한다면, Query Planning은 **그 의미를 어떤 순서와 데이터 소스를 통해 실행할 것인가**를 결정하는 단계이다.

Grounding된 질의는 실행 가능한 구조화된 QueryPlan으로 변환된다. QueryPlan에는 검색 대상, 필터 조건, 관계 탐색, 정렬 기준, 결과 개수 및 필요한 Retrieval Source가 명시된다.

현재 구현의 Planner는 deterministic한 방식으로 동작한다. 즉 LLM이 자유롭게 Tool 호출 순서를 생성하는 구조가 아니라, 미리 정의된 Query Semantic Model과 Routing Metadata에 따라 허용된 실행 경로를 구성한다.

예를 들어 다음 질의를 고려한다.

“특정 기업이 발행한 증권을 편입한 국내 ETF 중 순자산이 큰 상품을 알려줘.”

필요한 외부 관계 데이터와 조회 Capability가 준비된 경우, Planner는 이를 다음과 같은 실행 단계로 구성한다.

1. 기업을 식별하고 Graph에서 해당 기업이 발행한 Security 탐색
2. Graph에서 해당 Security를 편입한 ETF 후보 탐색
3. RDB에서 국내 상장 조건을 적용하고 비교 가능한 후보 ETF의 순자산 조회
4. 순자산을 기준으로 정렬
5. 최종 후보를 Evidence 대상으로 전달

그러나 실제 QueryPlan을 생성할 때는 **질의의 모든 의미적 조건이 실행계획에 포함되었는지**를 추가로 검증한다. 현재 구현에서는 지원되는 모든 material constraint가 하나 이상의 QueryStep 또는 내부 Transform에 대응되어야 하며, 질의에 있던 조건이 하나라도 누락되면 실행계획을 거부한다.

동시에 Planner가 사용자 질의에 존재하지 않는 조건을 임의로 추가하는 것도 허용하지 않는다. 따라서 본 시스템의 Planning Validation은 **No-Invention**과 **No-Omission**의 두 원칙을 함께 적용한다.

이는 실행 가능한 일부 조건만 남긴 뒤 부분적인 결과를 전체 질의의 답으로 반환하는 *silent semantic loss*를 방지하기 위한 것이다.

5.5 RDB / Graph / Semantic Retrieval

QueryPlan이 확정되면 각 QueryStep은 해당 의미를 가장 정확하게 처리할 수 있는 Retriever로 전달된다.

본 시스템은 하나의 검색 방식을 모든 질의에 적용하지 않고, RDB, Graph, Semantic Retrieval의 역할을 명시적으로 구분한다. 이는 기술세션자료에서 제시한 “Graph는 관계를 찾고, RDB는 숫자를 필터링하며, Vector는 의미를 검색한다”는 역할 분담을 실제 시스템에 적용한 것이다 [기술세션자료, 22쪽].

5.5.1 RDB Retrieval.

PostgreSQL 기반 RDB Retrieval은 정확한 값 조회와 수치 연산에 사용된다. 주요 대상은 다음과 같다.

- 상품 유형 및 정형 속성 필터링
- 출처별 지표 계약으로 허용된 수치 조회 및 정렬
- 비교 연산
- 정렬 및 Ranking
- 결과 개수 제한

RDB는 명시적인 데이터 Type과 연산자를 사용하여 수치 조회를 재현 가능하게 실행한다. 각 Field의 존재 여부와 Filter · Ranking 지원 여부는 별도로 검증한다.

5.5.2 Graph Retrieval.

Neo4j 기반 Graph Retrieval은 Entity 사이의 명시적인 관계를 따라가야 하는 질의에 사용된다.

예를 들어 기업-증권-ETF, 상품-운용사, 증권-발행사 등 복수 Entity 사이의 Relation을 연결해야 하는 경우 Graph Path를 이용하여 후보 Entity를 탐색할 수 있다.

특히 관계가 두 단계 이상 연결되는 질의에서는 각 관계를 독립적으로 검색하는 대신 앞 단계에서 얻은 Entity를 다음 Relation의 출발점으로 사용하는 bound multi-hop traversal을 수행한다.

5.5.3 Semantic Retrieval.

Semantic Retrieval은 전략 설명과 같이 서술형으로 제공되는 정보에서 질문과 관련된 원문을 찾는다. 현재 Corpus는 해외 ETP 데이터셋 PREF02N001의 cu_strategy에 대응하는 원천 Assertion으로 구성하며, 국내 ETF · 펀드 설명서나 뉴스 전반을 포괄하는 문서 집합은 아니다.

Supervisor는 질의의 표현과 조건에 따라 BM25 또는 Vector 검색을 선택한다. 기본 Vector 검색은 Token · Bigram · 금융 개념 Alias를 이용한 384차원 결정적 Hash Embedding과 Cosine Similarity를 사용한다. 이 점수는 전략 원문과 질의의 관련성을 나타내며, 금융상품의 수익성이나 적합성 점수로

사용하지 않는다.

검색 문서의 Canonical Entity와 원천 Assertion·Snapshot을 함께 전달하여 RDB 및 Graph의 결과와 동일 상품 기준으로 결합한다. 정형 조건이 포함된 질의에서는 해당 조건으로 제한된 후보 범위와 검색 결과의 교집합을 유지하며, 미해결 Entity의 문서는 답변 근거로 확정하지 않는다.

5.6 Evidence Construction

각 Retriever가 반환한 결과는 즉시 HyperCLOVA X에 전달되지 않는다. 서로 다른 저장소에서 얻은 결과를 하나의 **Evidence Bundle**로 재구성하는 과정이 먼저 수행된다.

Evidence Construction의 목적은 단순히 검색 결과를 나열하는 것이 아니라 최종 답변의 각 주장에 사용할 수 있는 **추적 가능하고 검증 가능한 근거 단위**를 만드는 것이다.

Evidence에는 다음과 같은 정보가 포함될 수 있다.

- 답변 대상 Canonical Entity
- 검색에 사용된 Source 및 Dataset
- 데이터의 기준 시점 또는 Snapshot
- 적용된 Filter 및 Ranking 조건
- 조회된 Metric과 단위
- Graph 탐색에 사용된 Relation Path
- 원천 데이터 또는 검색 결과에 대한 Provenance

예를 들어 “순자산이 가장 큰 ETF”라는 답을 생성하려면 상품명과 순자산 값만 존재하는 것으로 충분하지 않다. 해당 값이 어떤 데이터셋과 기준일에서 조회되었는지, 질의에 포함된 다른 필터 조건을 모두 만족했는지, 그리고 실제로 요청한 정렬이 수행되었는지까지 Evidence에 포함되어야 한다.

이러한 Evidence-First 구조를 통해 최종 생성 모델이 검색 결과를 다시 해석하면서 조건을 추가하거나 누락시키는 범위를 최소화한다.

5.7 Validation

Validation은 수집한 Evidence가 질의의 필수 조건과 실제 실행 결과에 부합하는지 확인하고, 최종 답변 생성으로 진행할 수 있는지를 결정한다. 조건 누락·추가, Entity·Relation 불일치, 필수 연산 미실행, 비교 제약 위반이나 근거 부족이 발견되면 Safe Response 경로를 사용한다.

이 판단은 Query-Plan, Plan-Execution, Execution-Evidence의 세 수준에서 이루어지며, 구체적인 검증 기준은 7.6절에서 설명한다. 조회가 정상 완료되어 조건에 맞는 상품이 없는 경우와, 조회 실패 또는 근거 부족으로 답을 결정할 수 없는 경우도 구분한다. 전자는 확인된 검색 범위의 결과를, 후자는 답변 제한 사유를 전달한다.

5.8 HyperCLOVA X 기반 최종 응답 생성

Validation을 통과한 Evidence를 최종 Answer Generator에 전달한다. 설정에 따라 HyperCLOVA X 답변기 또는 결정적 Evidence 답변기를 사용하며, 본 절은 HyperCLOVA X가 활성화된 경우의 역할을 설명한다.

본 시스템에서 HyperCLOVA X의 역할은 금융상품 데이터의 사실 여부를 새롭게 판단하거나, 직접 PostgreSQL 또는 Neo4j를 탐색하는 것이 아니다. 앞선 단계에서 이미 구조화·검색·검증된 Evidence를 바탕으로 사용자의 질문에 적합한 자연어 답변을 구성하는 **Grounded Answer Generation**을 담당한다.

즉 전체 역할은 다음과 같이 분리된다.

Ontology / Planner = What to retrieve
RDB / Graph / Semantic Retrieval = Find the evidence
Validator = Decide whether the evidence is sufficient
HyperCLOVA X = Explain validated evidence

Answer Generator에는 원천 데이터 전체가 아니라 질의와 관련된 검증된 Evidence Bundle만 제공한다. 이를 통해 Prompt Context를 제한하고, 모델이 데이터에 없는 사실을 임의로 생성할 가능성을 줄인다.

최종 답변에서는 가능한 경우 상품명, 주요 수치, 비교 결과와 함께 근거로 사용된 데이터 및 기준 시점을 명확히 제시한다. 반대로 Validator에서 충분한 근거를 확보하지 못한 경우에는 HyperCLOVA X를 이용해 추정 답변을 생성하지 않고 사전에 정의된 Safe Response 경로를 사용한다.

HyperCLOVA X는 efsubsec:query-understanding질의 의미 분석 보완에도 설정에 따라 사용될 수 있으며, 그 해석 결과 역시 Grounding과 검증을 거친다. 최종 응답 생성에서는 **Ontology-Grounded Retrieval**과 **Evidence Validation** 이후에 위치하는 **최종 자연어 추론 및 표현 계층**으로 사용된다.

이러한 구조를 통해 LLM의 자연어 이해·설명 능력은 활용하면서도, 금융상품의 검색, 수치 계산, 관계 탐색 및 답변 가능 여부의 판단은 구조화되고 검증 가능한 데이터 처리 계층에서 통제한다.

6. 주요 질의 유형 및 사용자 시나리오

금융상품에 대한 자연어 질의는 단순한 상품명 검색부터 수치 조건 비교, 관계 탐색, 구성종목 기반 탐색 및 자연어 설명 요청까지 다양한 형태로 나타날 수 있다.

본 시스템은 모든 질의를 동일한 검색 방식으로 처리하지 않고, 질의에서 요구되는 정보의 성격에 따라 RDB, Knowledge Graph, Semantic Retrieval을 선택하거나 조합한다.

다음 시나리오는 구현된 의미 계약과 처리 경로를 설명하는 예시이다. 실제 응답은 선택된 Snapshot, 준비된 외부 데이터, Entity Resolution 및 질의별 Capability 검증 결과에 따라 결정되며, 아래 예시를 서버 실측 결과로 제시하지 않는다.

대표적인 질의 유형과 주요 처리 방식은 다음과 같다.

6.1 조건 기반 금융상품 검색

가장 기본적인 사용자 시나리오는 여러 조건을 만족하는 금융상품을 찾는 것이다.

예를 들어 사용자는 다음과 같이 질문할 수 있다.

“미국에 투자하는 채권형 ETF를 알려줘.”

또는

“위험등급이 2등급 이하인 국내 펀드를 찾아줘.”

이러한 질의에서는 상품 유형, 투자지역, 자산 유형, 위험등급 등 사용자가 명시한 조건을 정확히 추출하고 해당 조건을 만족하는 상품만 검색해야 한다.

조건이 정형 Field와 직접 연결되는 경우에는 주로 RDB Retrieval을 사용한다. 예를 들어 질의

“미국에 투자하는 채권 ETF”

표 14: 대표 금융상품 질의 유형과 주요 처리 방식

질의 유형	주요 처리 방식	대표 정보
조건 기반 금융상품 검색	RDB 중심 Retrieval	상품 유형, 지역, 자산군, 위험등급 및 지원 상태 조건
지표 기반 Ranking	RDB + 검증	허용 Metric의 정렬, Top- <i>k</i> , 비교 가능성 확인
특정 기업 관련 상품 탐색	Graph + RDB	발행사, 운용사, 기업, 관련 상품 및 Metric
ETF 구성종목 기반 탐색	Graph / Holdings Data + RDB	ETF-Security 관계, 구성종목, ETF 속성
관계 기반 복합 질의	Graph + RDB Federated Retrieval	Multi-hop Relation, 후보 탐색, 정량 조건
자연어 설명형 질의	Semantic Retrieval + Structured Evidence	해외 ETP 전략 원문과 관련성 검색
데이터 부족·비교 불가 질의	Validation + Safe Response	Missing Evidence, Unsupported Capability, Ambiguity

는 개념적으로

ProductType = ETF
Region = US
AssetType = Bond

와 같은 구조화된 조건으로 변환할 수 있다.

이 과정에서 중요한 것은 자연어의 표현을 임의의 데이터 Field와 직접 연결하지 않는 것이다. 예를 들어 “채권”이라는 표현이 상품 자체로서 Bond를 의미하는지, ETF의 투자 자산 유형으로서 Bond를 의미하는지는 질의 Context를 함께 고려하여 판단해야 한다.

최종 답변에서는 조건에 부합하는 상품 목록과 함께 사용자가 판단에 활용할 수 있는 주요 속성을 제공한다.

6.2 지표 기반 Ranking

사용자는 상품 목록뿐 아니라 특정 지표를 기준으로 정렬한 결과를 요구할 수 있다. 현재 시스템은 출처와 모집단별로 허용된 지표의 Ranking을 수행하며, 이를 임의의 상품을 종합 평가하는 일반 비교 Intent와 구분한다.

“국내 ETF 중 순자산이 큰 상품 5개를 알려줘.”

이 질의에서는 국내 ETF 후보를 제한한 뒤, 해당 Source와 통화 범위에서 비교 가능한 AUM을 조회하고 내림차순 정렬과 Top-*k*를 적용한다. 조건에 해당하는 전체 후보 수와 지표를 확인할 수 있는 후보 수를 함께 관리하여, 일부 상품만 조회한 결과를 전체 모집단의 순위로 설명하지 않는다.

복수의 지표가 등장하더라도 나열 순서만으로 정렬 우선순위를 확정하지 않는다. 사용자가 우선 순위를 명시하고 모든 필수 정렬 Capability가 허용된 경우에만 그 순서를 실행계획에 보존한다. 우선순위가 모호하거나 연산이 지원되지 않으면 임의의 종합 점수나 순위를 만들지 않는다.

예를 들어 총보수 비교는 현재 cu_charge_rt의 Scale이 검증되지 않아 지원되는 Ranking 사례에 포함하지 않는다. 보수 비교 요청의 제한 처리는 6.7.3절에서 다룬다.

6.3 특정 기업 관련 금융상품 탐색

기업과 금융상품의 관계는 운용, 발행, 증권 편입에 따라 의미가 다르다. 예를 들어 다음 질의는 발행기관을 기준으로 채권을 찾는 요청이다.

“특정 기업이 발행한 채권을 찾아줘.”

기업을 Canonical Organization으로 식별한 뒤 Bond의 issuedBy 관계를 조회한다. 운용기관을 기준으로 ETF를 찾는 요청에는 managedBy를 사용한다. 동일한 Organization이라도 두 관계를 서로 대체하지 않는다.

반면 “삼성과 관련된 ETF” 처럼 기관과 관계가 모두 불명확한 표현은 운용기관인지, 발행 증권의 편입 기업인지 먼저 구분해야 한다. 표현만으로 Canonical Entity와 Relation을 확정할 수 없으면 가장 그럴듯한 관계를 임의로 선택하지 않고 모호성 또는 추가 조건을 안내한다.

6.4 ETF 구성종목 기반 탐색

특정 증권을 보유한 ETF를 찾는 질의는 ETF와 Security 사이의 HOLDS 관계를 사용한다.

“삼성전자 보통주(005930)를 편입한 ETF를 알려줘.”

이 예시에서는 국내 거래소 범위의 증권 식별자로 Security를 확인하고, 해당 Security를 대상으로 하는 HOLDS 관계를 역방향으로 조회하여 ETF 후보를 찾는다. 검색 결과에는 편입 관계의 Source와 Effective Date를 보존한다. 검증된 외부 보유내역과 양측 Entity가 준비된 범위에서만 관계를 사용한다.

기업 자체를 조건으로 하는 질의는 먼저 해당 기업이 발행한 Security를 찾아야 하므로 발행 관계가 추가로 필요하다. 이 관계와 수치 Ranking의 결합은 다음 절에서 설명한다.

6.5 관계 기반 복합 질의

관계 탐색과 수치 정렬이 결합되면 Ontology의 Entity · Relation · Identifier · Metric 모델이 하나의 실행 흐름에서 사용된다.

“삼성전자가 발행한 증권을 편입한 국내 ETF 중 순자산이 큰 3개를 알려줘.”

이 질의는 Organization에서 발행 증권을 찾고, 그 증권을 보유한 ETF 후보를 탐색한 뒤 정량 조건을 적용한다. 아래 표는 실제 관계 · 지표 계약에 따른 처리 구조를 나타내며, 특정 Snapshot의 후보 목록이나 실측 순위를 제시하는 것은 아니다.

핵심은 앞 단계의 Canonical Identifier를 다음 단계의 입력으로 전달하는 Bound Traversal이다. 발행기관과 편입 증권을 각각 독립적으로 검색하여 결합하지 않고, 연결된 Security와 ETF 후보를 RDB 조회까지 유지한다. 최종 Evidence에는 관계의 Fact · Source와 수치의 Observation · 기준일, 적용한 정렬 조건을 함께 남긴다.

필요한 관계나 지표가 없으면 해당 조건을 생략하지 않는다. 특히 “자회사가 발행한 증권”이라는 조건이 추가되면 현재 제공하지 않는 기업 간 자회사 관계가 필요하므로 미지원 관계로 처리한다.

6.6 자연어 설명형 질의

전략 설명처럼 서술형 원문에 기반한 질문은 Semantic Retrieval로 처리한다. 현재 검색 범위에 맞는 예시는 다음과 같다.

“이 해외 ETF의 원천 전략 설명에서 투자방식을 알려줘.”

표 15: 기업 발행 증권 · ETF 편입 · 순자산 질의의 의미 연결

질의 요소	의미와 실행 대상	확인할 근거
삼성전자 발행한 증권	Organization Resolution securityIssuedBy의 역방향 탐색	식별자 · Alias 후보와 해결 상태를 확인한다. 발행기관이 동일 Organization인 Security와 원천 발행 근거를 확인한다.
편입한 ETF	holds의 역방향 탐색	앞 단계의 Security를 실제로 보유한 ETF와 보유내역의 유효시점을 확인한다.
국내 ETF	후보 ID와 국내 상장 조건	투자노출 지역과 상장 범위를 혼동하지 않고 후보를 제한한다.
순자산 상위 3개	허용된 AUM 조회 · 내림차순 · Top-k	Source · 통화 · 기준시점 정책, 지표 누락과 정렬 가능한 후보 범위를 확인한다.
최종 답변	Evidence Bundle 및 Validator	관계 경로와 수치가 동일 후보를 가리키고 필수 조건이 실행되었는지 확인한다.

질문에서 특정한 상품의 identity를 확인하고, 해외 ETP 전략 Corpus에서 관련 원문을 조회한다. 상품의 이름 대신 앞선 대화의 “이 ETF”만 제시되어 대상을 확정할 수 없다면 상품명이나 식별자를 추가로 확인해야 한다.

검색 결과는 Canonical Product와 원천 Assertion을 기준으로 연결하며, 다른 상품의 전략이 섞이지 않았는지 검증한다. 검색 관련성 점수가 높다는 이유만으로 원문에 없는 특징을 사실로 확장하지 않는다. 국내 상품 설명서나 뉴스 등 현재 Corpus 밖의 정보는 동일한 기능으로 제공한다고 가정하지 않는다.

6.7 데이터 부족 · 비교 불가 질의 처리

금융상품 Agent에서는 모든 사용자 질문에 반드시 답을 생성하는 것보다 **현재 데이터와 기능으로 답할 수 없는 질문을 정확하게 식별하는 것이 중요하다.**

대표적인 경우는 다음과 같다.

6.7.1 필요한 데이터가 존재하지 않는 경우

예를 들어 사용자가 현재 KnowledgeBase에 없는 과거 시점의 데이터를 요청하면, 현재 Snapshot을 이용하여 해당 시점의 값인 것처럼 답변해서는 안 된다.

“2024년 말 기준으로 순자산이 가장 큰 ETF를 알려줘.”

현재 시스템이 해당 시점의 Snapshot을 보유하고 있지 않다면 해당 조건을 제거하여 현재 데이터를 기준으로 답하는 대신 요청한 시점의 데이터를 확인할 수 없음을 명시한다.

6.7.2 비교 기준이 불명확한 경우

다음과 같은 질의도 문제가 될 수 있다.

“상위 ETF 3개를 알려줘.”

여기서 “상위”가 순자산, 수익률, 보수, 거래량 중 무엇을 의미하는지 알 수 없다면 시스템이 임의로 Ranking Field를 선택해서는 안 된다.

이 경우에는 어떤 기준으로 순위를 원하는지 사용자에게 추가 조건을 요청할 수 있다.

6.7.3 지표 계약이 충족되지 않는 경우

“미국 주식형 ETF 중 총보수가 낮은 상품을 비교해줘.”

현재 ETP 보수율 원천값은 Scale이 검증되지 않아 비교용 Filter와 Ranking이 비활성화되어 있다. 따라서 보수 조건을 제거하여 순자산 순위로 대신 답하거나, 원천값을 임의로 퍼센트로 변환하지 않는다. 일반 비교 Intent의 지원 여부와 함께 요청을 검증하고, 현재 범위에서 비교할 수 없는 이유를 안내한다.

국내 ETP와 iShares의 1년 수익률도 산출 기준의 호환성이 확인되지 않아 하나의 모집단으로 통합 정렬하지 않는다.

6.7.4 지원하지 않는 의미가 포함된 경우

예를 들어

“가장 안전하고 앞으로 수익률이 높은 ETF를 추천해줘.”

와 같은 질의에는 주관적인 “안전성”, 미래 수익률 예측, 투자 추천과 같은 의미가 포함된다.

현재 데이터만으로 객관적으로 검증할 수 없는 의미를 임의의 Metric으로 치환하거나 LLM의 일반 지식으로 보완하지 않는다.

따라서 이러한 질의는

Unsupported / Insufficient Evidence → Safe Response

경로로 처리한다.

과제설명자료 역시 제공 데이터로 확인할 수 없는 질의가 평가에 포함되며, 해당 질문에 임의의 답변을 생성하는 것보다 “확인할 수 없음”을 명시하거나 필요한 조건을 요청하는 방식을 올바른 처리로 인정한다 [과제설명자료, 7쪽].

결과적으로 본 시스템에서의 사용자 경험은 모든 질문에 무조건 답하는 것이 아니라,

Answerable → Evidence-Based Answer

Ambiguous → Clarification

Unsupported / Insufficient → Safe Response

로 구분된다.

이를 통해 사용자는 시스템이 알고 있는 것과 현재 데이터로 확인할 수 없는 것을 명확하게 구분할 수 있다.

7. 신뢰성 및 검증

금융상품 Agent에서는 검색 결과가 존재한다는 사실만으로 답변의 신뢰성을 보장할 수 없다. 동일한 상품이라도 데이터의 출처와 기준 시점이 다를 수 있으며, Ontology의 의미와 실제 저장 데이터가 불일치하거나, 여러 저장소를 결합하는 과정에서 일부 질의 조건이 누락될 수 있기 때문이다.

따라서 본 시스템은 신뢰성을 LLM의 생성 단계에서만 해결해야 하는 문제가 아니라, **데이터 수집부터 Knowledge Construction, Query Planning, Retrieval, Evidence Construction, 최종 Answer Generation에 이르는 전체 Pipeline의 검증 문제**로 정의하였다.

본 시스템은 이를 위해 Source Provenance, Versioned Snapshot, Ontology-Data Consistency, Cross-Store Reconciliation, Numeric / Ranking Validation, Evidence Validation, Fail-Closed 정책을 다층적으로 적용한다.

7.1 Source Provenance

Provenance 검증은 4.1.6절에서 정의한 근거 연결이 실제 Fact와 조회 결과에 유지되는지를 확인하는 데 목적이 있다. Canonical 구축 단계에서는 증거 없는 확정 Fact, 잘못된 원천 연결과 관계의 Subject · Object 불일치를 검사하고, 질의 결과에는 Fact · Assertion · Source Record · Snapshot 식별자를 전달한다.

이를 통해 정규화된 값에서 원문 Field까지 확인할 수 있으며, 하나의 사실을 지원하는 여러 근거와 충돌하는 원천값을 구분한다. 외부 자료의 추가 검증 기준은 4.8절의 Provider · Hash · Effective Date 계약을 사용한다.

주최 측 원천과 외부 보완 자료의 Source를 분리하여 답변의 출처를 확인하고, 충돌 시 주최 측 데이터를 우선하는 과제의 데이터 활용 원칙을 적용할 수 있도록 한다 [과제설명자료, 6쪽].

7.2 Snapshot 및 Version 관리

금융 데이터는 시점에 따라 값이 변하기 때문에 동일한 질의를 언제 실행했는지가 결과의 의미에 영향을 줄 수 있다. 또한 Ontology, Mapping Logic 또는 Database Schema가 변경되면 동일한 원천 데이터에서도 서로 다른 Canonical Representation이 생성될 수 있다.

본 시스템에서는 이를 방지하기 위해 데이터를 단순한 “현재 상태”로 관리하지 않고 **Versioned Dataset Snapshot**으로 관리한다.

각 Snapshot에는 다음과 같은 정보가 함께 기록된다.

- 데이터 기준일(snapshot_date)
- 원천 데이터 Generation
- Ontology Version
- Semantic Mapping Version
- Transformer Version
- Database Schema Version
- Source Data Hash
- Schema Hash
- 원천 / 수용 / 격리 Row 수
- Validation 및 Reconciliation 상태

이를 통해 특정 답변이 어떤 Ontology와 어떤 데이터 생성 버전의 조합에서 생성되었는지를 재현할 수 있도록 한다.

또한 Snapshot은 생성 즉시 Production Query에 사용되지 않는다. 데이터 적재 및 검증 과정에 따라 STAGED, VALIDATING, READY, FAILED, RETIRED 등의 상태로 관리하며, **Reconciliation**을 통과하고 Row Count가 검증된 Snapshot만 **READY** 상태가 될 수 있도록 제한하였다.

따라서 Production Runtime은 단순히 가장 최근에 생성된 데이터를 선택하는 것이 아니라,

Correct Version + READY + Reconciliation Passed

조건을 만족하는 Snapshot만 사용한다.

동일한 Source Generation을 다시 적재하는 경우에도 변경이 없다면 기존 결과를 재사용할 수 있도록 idempotent rebuild를 지향하며, 실패한 적재가 기존 READY Snapshot을 훼손하지 않도록 Snapshot 단위로 상태를 격리한다.

배포 Artifact 역시 데이터와 Ontology, Semantic Index 등의 Hash를 검증한 후 사용하도록 구성하여 애플리케이션 코드 버전과 Knowledge Artifact 버전을 독립적으로 추적할 수 있도록 하였다.

7.3 Ontology-Data 정합성

Ontology는 Agent의 Semantic Layer로 사용되기 때문에 Ontology에 정의된 의미와 실제 데이터가 불일치하면 잘못된 Grounding이나 Graph Relation이 생성될 수 있다.

따라서 본 시스템은 Ontology 파일이 문법적으로 파싱되지만 확인하는 것이 아니라, **Ontology의 Semantic Contract와 Runtime Data Mapping의 정합성**을 함께 검증한다.

주요 검증 대상은 다음과 같다.

- 필수 Ontology Module의 존재 및 Parsing 여부
- Class 및 Concept Identifier의 유효성
- Alias와 Canonical Concept Mapping의 일관성
- Ontology Property와 Canonical Field Mapping의 일치
- Canonical Field와 실제 RDB Field Registry 간 정합성
- Relation의 Domain / Range 호환성
- Subclass 관계를 고려한 Relation Compatibility
- Source Data로 증명할 수 없는 Relation의 생성 여부

예를 들어 자연어의 “순자산”이 Ontology에서 특정 Property로 정의되어 있다면, 해당 Property는 Planner가 사용하는 Canonical Field와 연결되고, 다시 RDB의 실제 물리 Field까지 일관되게 연결되어야 한다.

Ontology Property → Canonical Field → RDB Field

이 Mapping 중 하나라도 존재하지 않거나 서로 다른 의미를 가리키면 해당 Field를 Runtime에서 사용하지 않는다.

Relation 역시 동일한 원칙을 적용한다. 예를 들어 특정 Relation의 Subject와 Object가 Ontology가 정의한 Domain / Range를 만족하는지 확인한 후에만 Canonical Relation 또는 Graph Edge로 활성화한다.

이 과정은 실제 Ontology 변경 과정에서도 중요한 안전장치로 사용된다. Ontology가 변경되었지만 Data Mapping과 Graph Projection이 아직 새로운 의미를 완전히 지원하지 않는 경우, 이전 Mapping을 추정하여 유지하는 대신 해당 Relation 또는 Concept을 unresolved 상태로 두도록 한다.

이는 기술세션자료에서 설명한 Ontology의 데이터 무결성 및 비즈니스 규칙 검증 역할과도 대응한다 [기술세션자료, 5-6쪽].

7.4 PostgreSQL-Neo4j 정합성

본 시스템에서 PostgreSQL과 Neo4j는 서로 독립적인 두 개의 진실 원천(Source of Truth)으로 사용되지 않는다.

PostgreSQL의 Canonical Data와 Relation을 검증된 기준 정보로 사용하고, Neo4j는 이 중 관계 탐색이 필요한 정보를 **Graph Projection** 형태로 구성한다.

따라서 Graph를 구축한 후에는 PostgreSQL과 Neo4j 사이의 Reconciliation을 수행한다.

검증 대상은 크게 다음과 같다.

- Canonical Entity와 Graph Node의 Identifier 일치
 - PostgreSQL Relation과 Neo4j Edge의 Relation Type 일치
 - Relation Subject / Object Identity 일치
 - Relation Type별 Record 수의 일치
 - Graph에만 존재하는 Orphan Node / Edge 여부
 - PostgreSQL에는 존재하지만 Graph Projection에서 누락된 관계 여부
 - 동일 Snapshot 및 Ontology Version에서 생성되었는지 여부
- 즉 Graph에서 다음과 같은 Edge가 존재한다면,

Entity A --[RELATION]--> Entity B

PostgreSQL의 Canonical Relation에서도 동일한 Subject, Relation Type, Object를 확인할 수 있어야 한다.

Graph는 관계 탐색을 빠르게 수행하기 위한 실행 표현(Execution Representation)이며, Canonical Data와 독립적으로 새로운 사실을 생성하지 않는다.

이를 통해 RDB Query와 Graph Query가 동일한 금융 Entity에 대해 서로 다른 결과를 반환하는 문제를 줄이고, 여러 저장소를 조합한 Federated Retrieval에서도 동일한 Canonical Identifier를 기준으로 결과를 결합할 수 있도록 한다.

Graph Projection 과정에서 Ontology의 Domain / Range 검증을 통과하지 못하거나 Source Provenance를 확보하지 못한 관계는 Neo4j에 적재하지 않는 것을 원칙으로 한다.

7.5 수치 및 Ranking 검증

수치 검증은 값이 숫자로 저장되었는지와, 요청한 연산이 금융적으로 허용되는지를 구분한다. 질의에서 해석한 값·단위·연산자를 보존한 뒤, 4.5절의 지표 계약으로 실행 가능성을 판단한다. 예를 들어 비율 원천값의 Scale이 검증되지 않았다면 임의로 퍼센트로 변환하여 Filter나 Ranking에 사용하지 않는다.

“국내 ETF 중 순자산이 큰 상품 5개를 알려줘.”

이 질의에서는 국내 ETF 모집단, AUM 내림차순과 결과 개수 제한을 보존한다. Planner가 정렬을 계획했지만 확인하지 않고, Execution Evidence에서 정렬이 실제 적용되었는지도 확인한다.

순위의 범위도 검증한다. 조건에 맞는 후보 수, 지표를 확인하여 정렬할 수 있는 후보 수, 지표 누락 수와 반환 후보를 구분한다. `filtered_total`, `rankable_total`, `missing_metric_total`, `ranked_candidate_ids` 등의 정보는 제한된 일부 결과를 전체 상품의 최상위 순위로 설명하지 않도록 사용된다.

복수 정렬은 사용자에게서 확인된 우선순위와 모든 필수 지표의 Capability가 유효할 때만 실행한다. 조건의 나열 순서에서 우선순위를 추정하거나, 지원되지 않는 조건 하나를 제거하여 나머지만 정렬하지 않는다.

단위·기간·통화·산출 기준의 비호환은 해당 비교를 제한한다. 관측 시점 차이 등은 지표별 정책에 따라 경고 또는 답변 제한으로 처리하며, 모든 차이를 일률적으로 동일하게 판단하지 않는다. 최종 Evidence에는 적용한 비교 범위와 정책을 함께 보존한다.

7.6 Evidence Validation

Evidence Validation은 앞선 모든 검증 결과를 최종 답변 생성 직전에 다시 확인하는 단계이다.

본 시스템에서는 Retrieval 결과가 존재한다고 해서 곧바로 “답변 가능”으로 분류하지 않는다. Evidence Validator는 사용자의 원래 질의와 실제 실행 결과를 대조하여 **질문의 의미가 검색 과정에서 보존되었는지**를 검증한다.

검증은 크게 세 수준에서 수행된다.

7.6.1 Query-Plan Consistency.

사용자 질의에서 추출한 모든 중요한 Constraint가 실제 QueryPlan에 포함되어 있는지 확인한다. Planner가 질의에 존재하지 않는 조건을 추가하는 **No-Invention** 위반과, 질의에 존재하는 조건을 누락하는 **No-Omission** 위반을 모두 차단한다.

7.6.2 Plan-Execution Consistency.

QueryPlan에 포함된 Filter, Relation Traversal, Ranking, Limit 등의 연산이 실제 Retriever 및 Executor에서 수행되었는지 확인한다.

예를 들어 Ranking을 계획했지만 실행 결과에 `ranking_applied=false`가 기록된 경우, 검색 결과가 존재하더라도 정상 답변으로 승인하지 않는다.

7.6.3 Execution-Evidence Consistency.

실행 결과가 최종 Evidence로 구성되는 과정에서 Entity, Relation, Metric, Unit, Source, Snapshot 등의 정보가 일치하는지 확인한다.

따라서 최종 Answer Generator에 전달되는 Evidence는

$$\text{Question} \equiv \text{Plan} \equiv \text{Execution} \equiv \text{Evidence}$$

의 의미적 일관성을 검사한 결과를 포함한다. 이 검사는 파서가 구조화한 제약과 실행 기록을 기준으로 하며, 임의의 자연어 표현을 항상 완전하게 이해한다는 보장은 아니다.

기술세션자료에서도 Federated Query의 최종 단계에서 여러 저장소의 결과를 통합한 뒤 무결성, 품질 등의 검증을 거쳐 Evidence로 구성하는 구조를 제시한다 [기술세션자료, 21쪽].

본 시스템에서는 이 검증 단계를 LLM 생성 이전의 강제적인 Gate로 사용함으로써, 필수 조건이나 근거가 부족한 결과가 생성 단계에 전달되지 않도록 제한한다.

7.7 Fail-Closed 전략

본 시스템의 최종 신뢰성 원칙은 **답할 수 있는 질문과 답할 수 없는 질문을 구분하는 것**이다.

일반적인 생성형 AI 시스템은 검색 결과가 부족하더라도 모델의 내부 지식을 이용해 그럴듯한 답을 생성할 수 있다. 그러나 금융상품 Agent에서는 이러한 동작이 오히려 위험할 수 있다.

따라서 본 시스템에서는 지원 여부가 불확실한 경우 가능한 범위만 실행하여 답을 생성하는 Fail-Open 방식 대신, **전체 질의를 신뢰성 있게 만족할 수 있을 때만 정상 답변을 생성하는 Fail-Closed 전략**을 사용한다.

다음과 같은 상황에서는 정상 Answer Generation을 중단한다.

- 중요한 자연어 Clause가 해석되지 않은 경우
- Entity 또는 Relation이 모호하게 Resolution된 경우
- Ontology에서 지원하지 않는 Concept 또는 Relation인 경우
- QueryPlan에서 필수 Constraint가 누락된 경우

- 필요한 Retrieval Capability가 구현되어 있지 않은 경우
- 필수 Ranking 또는 Comparison이 실제 실행되지 않은 경우
- 수치의 단위 또는 비교 기준이 호환되지 않는 경우
- 데이터 또는 Provenance가 충분하지 않은 경우

이 경우 시스템은 해당 조건을 제거하고 나머지 조건만으로 결과를 생성하지 않는다.
예를 들어

“AI와 친환경 산업에 모두 투자하는 ETF”

라는 질의에서 두 Semantic Condition의 AND 관계를 정확히 보장하는 검색 기능이 없다면, “AI ETF” 또는 “친환경 ETF” 중 하나만 검색하여 결과를 반환하지 않는다.

마찬가지로 “상위 3개”라는 표현에서 어떤 Metric을 기준으로 상위를 정의해야 하는지 결정할 수 없는 경우에도 임의의 Ranking Field를 선택하지 않는다.

대신 사용자에게 현재 데이터 또는 실행 기능으로는 해당 질의를 신뢰성 있게 확인할 수 없음을 알리는 Safe Response를 반환한다.

과제설명자료에서도 제공 데이터로 확인할 수 없는 질의에 임의의 답변을 생성하는 경우 감점하며, “확인할 수 없음”을 명시하거나 필요한 조건을 요청하는 방식을 올바른 처리로 인정하고 있다 [과제설명자료, 7쪽].

따라서 본 시스템의 목표는 모든 질문에 가능한 답을 생성하는 것이 아니라,

Answer Only What Can Be Verified

라는 원칙 아래 검증 가능한 범위에서는 정확하게 답하고, 검증할 수 없는 범위에서는 명시적으로 답변을 제한하는 것이다.

이러한 Fail-Closed 정책은 Ontology Grounding, Query Planning, Retrieval, Evidence Validation 전 단계에서 적용되며, 근거가 부족한 질의의 최종 생성 단계 진입을 차단한다. HyperCLOVA X에는 검증된 근거만 전달하여 환각 가능성을 줄인다. 현재 구현은 생성 전 Evidence를 검증하며, 생성된 문장 전체를 Canonical Fact와 다시 대조하는 별도의 후단 사실 검증까지 수행하는 것은 아니다.

7.8 검증 근거와 범위

최종 제출 후보의 검증 기록을 기준으로 확인된 결과와 그 해석 범위를 표 16에 정리하였다. 기준 코드는 286334eec6e90320bd0178a10b396abeec12cd35이며, 데이터 세대는 260824이다.

따라서 본 보고서는 모델과 실행 계약의 구현·대사 근거를 제시하며, Ontology 적용 전후의 질의 정답률 향상이나 HyperCLOVA 응답 품질을 위 수치만으로 주장하지 않는다. 6장의 시나리오는 처리 구조의 설명이며, 실제 결과를 평가할 때에는 코드·데이터 Version과 함께 질의, 기대 조건, 실행 Evidence, 최종 응답 및 답변 제한 사유를 기록해야 한다.

8. 기대 효과 및 확장성

본 제안의 목적은 단순히 금융상품 검색 정확도를 높이는 데 있지 않다. Ontology를 중심으로 자연어 질의, 금융상품 데이터, 여러 검색 방식 및 최종 답변 생성 과정을 연결함으로써, **금융상품 탐색의 편의성, 답변의 검증 가능성, 데이터 확장성 및 서비스 확장성**을 동시에 확보하는 것을 목표로 한다.

표 16: 최종 제출 후보의 점검 결과와 해석 범위

점검 대상	기록된 결과	해석 범위
Canonical 구축	FinancialProduct 35,180개, FundShareClass 16,574개, Security 753개	원천과 변환 규칙에 따른 개체·grain 대사 결과이며 질의 정답률은 아니다.
관계·Graph 대사	HOLDS 3,281건, SECURITY_ISSUED_BY 571 건; PostgreSQL과 Neo4j의 누락·추가 0건	두 저장소의 관계 수와 Identifier 일치율 확인한다. 관계 사실의 원천 품질 평가는 별도 계약에 따른다.
수치 관측	ONE_YEAR_RETURN Observation 8,420건	저장된 관측 수이며 모든 Source 간 통합 비교 가능성을 뜻하지 않는다.
코드 테스트	310개 통과, 99개 건너뛸	실행한 테스트 범위의 회귀를 확인한다. 건너뛴 외부 의존 경로와 실제 답변 정확도는 별도 검증이 필요하다.
서비스 확인	/live HTTP 200	프로세스 생존 확인이며 데이터 준비 상태나 금융 응답 품질의 증거는 아니다.

특히 Ontology와 Canonical Data Model을 개별 데이터 Source나 특정 LLM으로부터 분리하였기 때문에, 새로운 금융상품, 데이터 공급자 또는 사용자 Context가 추가되더라도 전체 시스템을 다시 설계하기보다 기존 Semantic Layer를 확장하는 방식으로 대응할 수 있다.

8.1 금융상품 탐색 편의성

기존 금융상품 탐색에서는 사용자가 상품별 데이터 구조와 금융 용어를 사전에 이해하고, 여러 조건을 직접 조합하여 검색해야 하는 경우가 많다.

예를 들어 사용자가

“국내 상장 ETF 중 미국 주식에 투자하는 상품을 순자산이 큰 순서로 3개 알려줘.”

라고 질문하고 싶더라도, 전통적인 검색 방식에서는 상장 범위, 상품 유형, 투자지역, 자산군, 순자산과 같은 여러 조건을 각각의 검색 Field에 맞추어 입력해야 한다.

제안 시스템에서는 사용자가 이러한 내부 데이터 구조를 알지 못하더라도 자연어만으로 금융상품을 탐색할 수 있다.

Agent는 질의에서 상품 유형, 조건, Relation, Metric, Ranking 기준 등을 추출한 뒤 Ontology를 통해 실제 데이터와 연결한다.

따라서 사용자는

Database Schema 중심 탐색 → 사용자 의도 중심 탐색

으로 금융상품 검색 방식을 전환할 수 있다.

특히 단순 상품명 조회뿐 아니라 조건 검색, Ranking, 관계 탐색, 자연어 설명형 질의를 하나의 인터페이스에서 처리할 수 있다는 점에서 금융상품 탐색의 진입 장벽을 낮출 수 있다.

8.2 근거 기반 답변 신뢰도 향상

금융상품 Agent에서 중요한 것은 자연스러운 문장을 생성하는 것뿐 아니라 **왜 해당 답변이 생성되었는지를 확인할 수 있는가**이다.

제안 시스템은 검색 결과를 즉시 LLM에 전달하지 않고, Evidence Construction 및 Validation 단계를 통해 최종 답변에 사용될 정보를 별도로 검증한다.

이를 통해 최종 답변은 상품명이나 수치 결과뿐 아니라 필요한 경우 다음 정보를 함께 제시할 수 있다.

- 어떤 금융상품 데이터가 사용되었는가
- 어떤 기준 시점의 데이터인가
- 어떤 Filter 및 Ranking 조건이 적용되었는가
- 어떤 Entity 및 Relation을 통해 결과가 도출되었는가
- 어떤 Source가 해당 사실을 뒷받침하는가

또한 충분한 Evidence를 확보할 수 없는 질의는 LLM의 내부 지식으로 보완하여 답변하지 않고 Safe Response로 처리한다.

따라서 시스템의 신뢰성은 LLM이 얼마나 그럴듯한 답변을 생성하는가에만 의존하지 않고,

Question → Plan → Retrieval → Evidence → Validation

과정 전체에서 확보된다.

이는 금융상품 질의응답에서 환각 가능성을 줄이는 것뿐 아니라, 답변 결과를 다시 검토하거나 재현할 수 있다는 측면에서도 실무 활용 가능성을 높인다.

8.3 금융상품 간 의미적 연결

전통적인 금융상품 검색에서는 각 상품을 하나의 독립된 Record로 조회하는 경우가 많다. 그러나 실제 금융상품은 기업, 운용사, 발행사, 지수, 증권, 통화 및 상품분류 등 다양한 Entity와 연결되어 있다.

Ontology와 Knowledge Graph를 활용하면 이러한 연결을 단순한 문자열 Field가 아니라 명시적인 Relation으로 표현할 수 있다.

예를 들어

ETF → AssetManager

또는

ETF → Index

와 같은 관계를 동일한 Semantic Model에서 탐색할 수 있다.

데이터가 추가되면 이러한 관계는 더 복잡한 Multi-hop Query로 확장될 수 있다.

Product → Security → Company → Related Product

와 같은 탐색이 가능해지면, 단순히 “조건에 맞는 상품을 찾는 것”을 넘어 **금융상품 사이의 의미적 관계를 탐색하는 Agent**로 발전할 수 있다.

기술세션자료 역시 단일 Retriever만으로 처리하기 어려운 복합 질의에서 Graph와 RDB를 함께 활용하는 Federated Retrieval의 필요성을 제시한다 [기술세션자료, 20-23쪽].

따라서 Knowledge Graph의 가치는 Graph DB 자체를 사용하는 데 있는 것이 아니라, 정형 데이터만으로 드러나기 어려운 금융 Entity 사이의 관계를 자연어 질의에서 직접 활용할 수 있다는 데 있다.

8.4 신규 금융상품 및 데이터 공급자 확장

제안 시스템에서는 원천 데이터의 Schema를 Agent의 Query Logic과 직접 결합하지 않는다.
새로운 데이터는 먼저

Source Data → Canonical Data Model → Ontology Mapping

과정을 거친 뒤 기존 Knowledge Layer에 연결된다.

따라서 향후 ELS 등 현재 모델에 포함되지 않은 금융상품이나 새로운 시장·기관에서 제공되는 데이터가 추가되더라도, 기존 Agent의 전체 질의 처리 구조를 다시 작성할 필요가 없다.

새로운 금융상품의 경우에는 기존 Ontology의 상위 Class를 재사용하거나 필요한 Subclass 및 Property를 추가할 수 있으며, 새로운 데이터 공급자는 Source-specific Transformation과 Canonical Mapping을 추가하는 방식으로 수용할 수 있다.

이러한 구조는

Source-specific Logic ≠ Agent Reasoning Logic

이라는 분리를 가능하게 한다.

즉 데이터 공급자가 달라져도 사용자에게 제공되는 금융 개념과 질의 방식은 Ontology를 중심으로 일관되게 유지할 수 있다.

또한 Source와 Snapshot을 별도로 관리하기 때문에 여러 데이터 공급자가 동일한 사실을 제공하는 경우에도 기존 Canonical Fact를 중복 생성하기보다 복수의 Evidence를 연결하는 방식으로 확장할 수 있다.

8.5 실시간 데이터 연계

현재 제안 시스템은 검증된 Snapshot을 기반으로 재현 가능한 금융상품 질의응답을 제공하는 것을 우선한다.

그러나 금융상품 데이터는 순자산, 가격, 수익률, 구성종목 등 시간에 따라 지속적으로 변하는 특성을 가진다. 따라서 향후에는 외부 금융 API 또는 실시간·주기적 데이터 Feed를 현재의 Snapshot Architecture와 결합할 수 있다.

이 경우에도 실시간 데이터를 LLM에 직접 전달하는 방식보다는

**External Feed → Validation → Versioned Snapshot → Canonicalization →
Knowledge Layer**

의 기존 원칙을 유지하는 것이 중요하다.

특히 금융상품의 Metric은 값 자체뿐 아니라 관측 시점과 기준 기간이 중요하기 때문에, 실시간 확장에서도 `observedAt`, `Source`, `Unit`, `Metric Definition` 등의 정보를 함께 관리해야 한다.

또한 실시간성과 재현성은 서로 상충할 수 있으므로, 향후에는

- 최신 데이터 조회가 필요한 질의,
- 고정된 기준일의 데이터가 필요한 질의

를 구분하여 처리하는 방식으로 확장할 수 있다.

이를 통해 현재의 Snapshot 기반 안정성을 유지하면서도 시장 변화에 민감한 금융상품 질의에 더 최신의 정보를 제공할 수 있다.

8.6 데이터 품질 분석 및 보완 가능성

본 시스템은 2026년 8월 24일 재배포된 데이터를 기준으로 국내 ETP, 해외 ETP, 공모펀드, 국내 채권의 결측 및 이상값을 분석하였다. 분석 결과, 원천 데이터의 공란은 모두 동일한 의미의 결측이 아니었으며, 실제 정보가 제공되지 않은 경우뿐 아니라 상품 구조상 해당 속성이 적용되지 않는 경우, 특정 상태를 나타내는 Sentinel 값, 기준일 당시 아직 산출되지 않은 값 등이 함께 존재하였다. 따라서 본 시스템은 결측을 일괄적으로 대체하지 않고, 그 의미와 발생 원인을 구분하여 질의 가능 여부와 데이터 검증에 활용하였다.

상품군별로도 결측의 성격은 서로 다르게 나타났다. 국내 ETP에서는 Refinitiv 연계 정보가 572개 상품에서 누락되어 있었고, 현재 판매 ETF 1,160개 중 945개는 총보수율이 제공되지 않았으며, 현재 판매 ETN 374개는 구조화된 기초지수 정보가 부족하였다. 해외 ETP에서는 14개 상품의 가격·판매 상태 등 핵심 정보가 함께 누락되었고, 336개 상품은 오래된 종가를 보유하고 있었으며, 2,931개 상품에서는 유효한 기초지수를 확인하기 어려웠다. 공모펀드는 일간 기준가·순자산 정보가 14,263개 클래스에서 제공되지 않았고, 1주 수익률은 전체 행에서 결측이었으나, 장기 수익률의 일부 결측은 상품의 운용기간이 충분하지 않아 발생한 구조적 결측으로 확인하였다. 국내채권에서는 21,248개 행에 개별 판매조건이 없었지만, 이는 구매 불가를 의미하지 않고 SaleLot 정보가 제공되지 않은 경우에 해당하였다. 반면 회사채의 신용등급 누락 926건, 발행기관 누락 149건, 채권종류 누락 152건 등은 향후 보완 가치가 있는 정보로 구분하였다.

결측의 현황을 파악한 이후에는 원천 데이터의 다른 필드와 별도의 조사 결과를 활용하여 일부 항목이 향후 어느 수준까지 보완될 수 있는지도 검토하였다. 특히 국내 ETP의 Refinitiv 연계 결측 572건에 대해서는 별도의 분석 산출물에서 보완 후보값을 생성하고 그 산출 근거를 함께 기록하였다. 구체적인 검토 결과는 Table 17와 같다.

표 17: 국내 ETP Refinitiv 결측정보의 보완 가능성 검토

항목	확인 결과	검토 방식
pd_isin_cd	571건	pd_itm_no의 ISIN 형식과 checksum을 검증하여 대응 가능한 값을 확인하였다.
pd_ticker	572건	pd_itm_no_ma의 시장구분 접두어를 제거하여 단축코드 후보를 산출하였다.
pd_ric	572건	시장단축코드에 .KS를 결합하여 후보값을 산출하였으나, LSEG에서 직접 검증된 값은 아니므로 추정값으로 구분하였다.
ref_base_index	24건	원천 데이터의 cu_base_index에 정확한 지수명이 존재하는 경우에 한해 대응값을 확인하였다.
ref_geo_focus, ref_ast_type	각 572건	기존 투자지역과 투자자산 유형을 대응되는 영문 분류체계로 매핑하였다.
ref_base_dt	0건	Refinitiv 자체의 정보 기준일이므로 다른 날짜를 임의로 대체하지 않았다.

이와 같은 결측 분석을 바탕으로, 본 프로젝트에서는 단순히 누락 여부를 식별하는 데 그치지 않고 각 항목에 대해 활용 가능한 내부 필드와 외부 공개정보를 대조하여 **실제 적용 가능한 보완 후보값과 산출 규칙까지 구체화하였다**. 이를 통해 외부 정보와의 연계를 통해 보완할 수 있는 항목과 원천 의미를 보존하기 위해 기존 상태를 유지해야 하는 항목을 구분하였으며, 향후 데이터 확장에 활용할 수 있는 보완 후보군을 별도로 확보하였다.

다만 확인된 보완 결과를 평가용 Database에 직접 통합하는 대신, **평가 데이터와 외부 보완 데**

이터를 분리하여 관리하는 방식을 채택하였다. 과제 설명에서는 제공 데이터를 기준으로 질의에 답하도록 하고 있으며, 외부 데이터와 제공 데이터가 상충하는 경우에는 주최 측 제공 데이터를 정답 기준으로 사용하도록 안내하고 있다 [과제설명자료, 기술세션자료]. 이에 따라 실제 검색, 비교 및 답변 생성에는 재배포된 원천 Database를 일관된 *Source of Truth*로 유지하고, 결국 분석을 통해 확인한 보완 후보값은 평가 결과에 영향을 주지 않는 별도의 확장 데이터로 관리하였다. 이는 보완 작업의 결과를 배제한 것이 아니라, **평가 단계의 재현성과 사실 기준의 일관성을 유지하면서 향후 서비스 확장에 필요한 데이터 연계 가능성을 함께 확보하기 위한 설계**이다.

이러한 구조를 통해 현재 평가 환경에서는 제공 데이터의 범위를 벗어나는 정보를 임의로 사용하지 않으면서도, 실제 서비스 단계에서는 검증된 외부 데이터를 출처와 기준일과 함께 연계할 수 있다. 향후 기초지수, 비용, 위험 및 최신 시장정보 등이 추가되면 현재 데이터의 한계로 처리하기 어려운 상품 비교 및 탐색 질의의 범위를 확대하고, 데이터 부족으로 인한 답변 불가 영역도 줄일 수 있다. 나아가 상품 정보의 완전성과 최신성이 향상되면 사용자별 금융정보와 결합하여 맞춤형 Agent로 확장하기 위한 기반으로 활용할 수 있다.

8.7 사용자 맞춤형 Agent로의 발전

현재 시스템은 질문에 명시적으로 나타난 상품 조건과 사용자의 요구를 바탕으로 금융상품을 탐색하는 것을 중심으로 한다. 따라서 동일한 질문에는 원칙적으로 동일한 금융상품 데이터와 조건이 적용된다.

향후에는 금융 마이데이터와 연계하여 사용자 맞춤형 금융상품 탐색 Agent로 확장할 수 있다. 금융 마이데이터는 은행, 카드사 등 여러 금융기관에 분산된 개인 신용정보를 한곳에서 파악하고 관리할 수 있도록 지원하는 제도이며, 이러한 통합 데이터를 활용한 개인화 서비스의 강화는 금융 서비스의 주요 경쟁 요소로 언급되고 있다 [빅데이터뉴스].

본 시스템에서는 사용자의 동의하에 제공되는 보유자산, 관심 상품, 투자 가능 시장, 투자기간 및 선호 조건과 같은 정보를 별도의 사용자 Profile Layer에 명시적인 Context로 구조화할 수 있다. 이렇게 구성된 사용자 Context를 금융상품 Knowledge Layer와 연결하면, 상품 자체의 특성과 사용자의 조건을 함께 고려하는 개인화된 탐색이 가능해진다.

예를 들어 동일한

“채권형 상품을 찾아줘.”

라는 질문이라도, 사용자가 현재 보유한 상품, 관심을 보인 시장, 투자 가능한 기간 등에 따라 검색해야 할 후보 범위와 우선적으로 설명해야 할 정보가 달라질 수 있다.

다만 사용자 맞춤형 서비스에서도 LLM이 사용자의 성향이나 투자 목적을 임의로 추측하여 금융상품을 추천하도록 하는 방식은 지양해야 한다. 사용자 Context는 사용자가 직접 입력한 정보, 동의하에 제공된 마이데이터, 시스템이 질의로부터 확인한 조건을 구분하여 관리해야 한다.

또한 어떤 사용자 정보와 상품 조건이 검색, 필터링 및 비교에 사용되었는지를 추적할 수 있어야 한다. 이를 통해 개인화의 수준을 높이면서도 결과의 근거와 설명 가능성을 유지할 수 있다.

개념적으로는 다음과 같이 확장할 수 있다.

User Query + Explicit User Context + Financial Ontology + Verified Product Evidence → Personalized Financial Agent

여기서 Explicit User Context에는 사용자가 직접 제시한 조건뿐 아니라, 사용자의 동의를 받아 제공된 마이데이터 중 해당 질의에 필요한 정보가 포함될 수 있다.

특히 현재 구축한 Ontology와 Canonical Data Model은 금융상품 자체의 의미를 사용자 Profile과 분리하여 관리한다. 따라서 향후 마이데이터 기반 개인화 기능을 추가하더라도 기존 금융상품

Knowledge Layer를 재사용하면서, 사용자 Context의 Mapping과 적용 조건을 추가하는 방식으로 확장할 수 있다는 장점이 있다.

궁극적으로는 단순한 금융상품 검색을 넘어, 사용자가 보유한 금융정보와 현재 시장의 상품정보를 연결하면서도 어떤 데이터와 조건을 근거로 결과가 도출되었는지를 설명할 수 있는 개인화 금융 Agent로 발전할 수 있다.

9. 결론

본 제안은 금융상품 질의응답을 단순한 LLM 기반 검색 문제로 보지 않고,

**Natural Language → Explicit Financial Semantics → Verifiable Evidence →
Grounded Answer**

의 문제로 정의하였다.

금융상품 데이터는 상품군과 Source에 따라 구조와 의미가 다르고, 사용자의 자연어 질의는 Entity, Relation, Metric, 수치 조건 및 Ranking과 같은 여러 의미 요소를 동시에 포함할 수 있다.

이에 본 시스템은 Ontology와 Canonical Data Model을 전체 시스템의 공통 Semantic Layer로 사용하고, 자연어 질의를 명시적인 금융 의미에 Grounding한 뒤 RDB, Knowledge Graph 및 Semantic Search를 질의의 특성에 따라 조합하는 **Ontology-Grounded Federated Agent Architecture**를 제안하였다.

전체 과정은

Define → Connect → Retrieve → Validate → Answer

의 흐름으로 요약할 수 있다.

먼저 Ontology를 통해 금융상품의 개념, Field, Relation 및 Constraint를 정의하고, 서로 다른 Source의 데이터를 Canonical Entity와 Fact로 연결한다. 이후 사용자 질의는 Ontology Grounding과 Query Planning을 거쳐 RDB, Graph, Semantic Retrieval에 분배되며, 각 저장소에서 수집한 결과는 Evidence로 통합된다.

특히 본 시스템은 검색 결과가 존재한다는 이유만으로 즉시 최종 답변을 생성하지 않는다. 질문의 모든 중요한 조건이 실행되었는지, Entity와 Relation이 의미적으로 일치하는지, 수치 및 Ranking 조건이 실제로 적용되었는지, 충분한 Source Evidence가 존재하는지를 검증한 후에만 최종 Answer Generation 단계로 전달한다.

충분한 Evidence가 확보되지 않은 경우에는 일부 조건을 임의로 제거하여 불완전한 답을 생성하는 대신, Safe Response를 반환하는 **Fail-Closed 전략**을 적용한다.

최종 응답 생성에서 HyperCLOVA X는 구조화되고 검증된 Evidence를 사용자가 이해할 수 있는 자연어 답변으로 변환하는 최종 생성 계층으로 활용된다.

본 시스템의 핵심 차별점은 RDB, Graph, Semantic Search 또는 LLM이라는 개별 기술을 사용하는 것 자체에 있지 않다. 핵심은 이들을 하나의 Ontology와 Canonical Semantic Model을 중심으로 연결하여,

질문의 의미가 데이터 검색과 최종 답변까지 일관되게 보존되도록 하는 것

에 있다.

향후 새로운 금융상품, 외부 데이터 공급자 및 실시간 금융 데이터가 추가되더라도 기존 Semantic Layer를 중심으로 KnowledgeBase와 Retrieval Capability를 단계적으로 확장할 수 있다. 나아가 사용자의 동의하에 금융 마이데이터와 사용자 Profile Layer를 연계한다면, 보유자산과 투자조건을

명시적인 Context로 활용하면서도 어떤 정보와 조건을 근거로 결과가 도출되었는지를 설명할 수 있는 개인화 금융 Agent로 발전시킬 수 있다.

이를 통해 본 제안은 현재의 금융상품 분석 및 질의응답을 넘어, **검증 가능한 금융 지식을 기반으로 사용자의 금융 의도를 이해하고 설명하는 확장 가능한 Financial Product Agent**의 기반을 제공한다.

참고문헌

[과제소개자료] 미래에셋증권, 「제10회 2026 미래에셋증권 AI Festival 과제 소개 - 금융상품 Agent (배포용)」, 2026.07.

[과제설명자료] 미래에셋증권, 「제10회 2026 미래에셋증권 AI Festival 과제 소개 - 금융상품 Agent (참가자공유용)」, 2026.07.

[기술세션자료] 이우람, 「2026 미래에셋증권 AI Festival Technical Session」, 미래에셋증권 AI Engineering 본부, 2026.

[빅데이터뉴스] 임예린, 「미래에셋증권, 금융 마이데이터 브랜드평판 9월 1위...현대카드·하나은행 순」, 빅데이터뉴스, 2026.09.01.

[디지털데일리] 이호연, 「금융권 AI 경쟁, 생성형 넘어 ‘에이전트’로...2030년 도입률 81% 전망」, 디지털데일리, 2026.08.12.

[FIBO] EDM Council, 「Financial Industry Business Ontology (FIBO)」, GitHub, <https://github.com/edmcouncil/fibo>, 2026.

A. 용어 및 약어

Ontology 금융상품 도메인의 개념, 속성, 관계 및 의미 제약을 정의하는 지식 모델

Knowledge Graph

Ontology 구조 위에 실제 엔티티와 관계 데이터를 연결한 그래프

Canonical Data Model

서로 다른 원천 데이터 구조를 공통 의미 체계로 정규화한 데이터 모델

Federated Query

질의를 여러 저장소에 분해·라우팅하고 결과를 통합하는 검색 방식

Evidence Bundle

최종 답변 생성을 위해 검증된 근거를 구조화한 묶음

Fail-Closed

충분한 근거가 없을 때 추정 답변 대신 명시적으로 응답을 제한하는 전략